

# Mastering Large Language Models: From Basics to Advanced Fine-Tuning

## Introduction to Large Language Models (LLMs)

Large Language Models (LLMs) have revolutionized how machines understand and generate human language. These are AI models with **massive numbers of parameters** trained on extensive text datasets, enabling them to perform a wide range of language tasks with minimal task-specific training <sup>1</sup>. In daily life, LLMs power virtual assistants, chatbots, translation services, and content generation tools. For example, *ChatGPT* (an LLM-based chatbot) can draft emails or answer questions in natural language, and *BERT*-based models can classify the sentiment of product reviews with high accuracy. This introduction lays the groundwork for understanding what LLMs are, why they are useful, and how they differ from earlier approaches in Natural Language Processing (NLP).

**Real-Life Example – LLM in Action:** Imagine a tech support chatbot on an e-commerce website. It uses an LLM to understand customer queries and provide helpful answers. Without any explicit if-else rules, the LLM can interpret a question like “I ordered a phone, but it hasn’t arrived. What should I do?” and generate a coherent response. This flexibility comes from training on vast amounts of conversational data. Throughout this book, we will explore how such capabilities are developed, from basic principles to advanced fine-tuning techniques that adapt an LLM to specific real-world tasks.

## Chapter 1: Foundations of Natural Language Processing

Before diving into LLMs, it’s important to understand the **foundations of NLP** that led to their development. Early NLP methods dealt with text using simpler statistical models and rule-based systems:

- **Word Counts and Frequency:** The most basic approach to understanding text is counting words. For example, a **bag-of-words model** represents a document by the frequency of each word, disregarding grammar and word order entirely. This approach can tell you which words are common in a text but loses context.
- **N-Gram Language Models:** Before neural networks, language modeling often relied on *n-grams* – sequences of  $n$  words. An  $n$ -gram model predicts the next word by looking at the previous  $n-1$  words. For instance, a **bigram** (2-gram) model might predict the word “cake” after “chocolate” because the phrase “chocolate cake” frequently appears in the training text. These models learn probabilities of word sequences from a corpus. While  $n$ -grams can capture some local structure of language, they struggle with long-range dependencies. If a sentence is long, an  $n$ -gram model has no memory beyond its  $n-1$  window. This leads to *the curse of dimensionality* – as  $n$  grows, the number of possible sequences explodes, making it infeasible to gather enough statistics for all of them.
- **Limitations:** Traditional models couldn’t handle things like understanding context beyond a few words or generalizing to unseen word combinations. For example, an  $n$ -gram model might have seen “New York” and “New Delhi” in training, but not “New Phone” as a phrase – it would struggle to

predict “Phone” after “New” if that exact bigram wasn’t observed. These models also treated each word as an atomic unit, ignoring similarities in meaning (e.g., “big” and “large” were unrelated to an n-gram model if treated as separate tokens).

**Real-Life Example – Evolution of Spell Check:** Early spell-checkers used simple dictionaries and rules (e.g., if a word is not in the dictionary, flag it). More advanced ones used n-gram probabilities: if a rare word appears that doesn’t fit common sequences (like “hte” in “hte cat”), it suggests a correction (“the”). However, these systems would often fail for longer context issues. For instance, they might not catch a wrong word usage like “I ate dessert in the desert” because each word is spelled correctly. This demonstrated the need for models that understand context, a gap eventually filled by more sophisticated NLP techniques and LLMs.

## Chapter 2: The Road to Large Language Models

The journey from basic NLP to LLMs went through several key innovations. This chapter highlights the **evolution of approaches**, each addressing limitations of the previous methods:

### 2.1 Emergence of Machine Learning in NLP

In the 2000s, machine learning algorithms like **Naive Bayes** and **Support Vector Machines (SVM)** were applied to text classification tasks (like spam detection or sentiment analysis). Text had to be converted into numerical features (often using bag-of-words or n-grams as features). These algorithms could weigh the importance of words for a task – for example, Naive Bayes could learn that “free” and “win” are strong indicators of spam emails. However, these models still couldn’t truly *understand* language; they relied on manually crafted features or simple frequencies.

### 2.2 Introduction of Word Embeddings

A major breakthrough in representing text came with **word embeddings**. Methods like **Word2Vec** and **GloVe** (mid-2010s) learned vector representations for words such that words with similar meanings were close in the vector space. For example, *king* might be near *queen*, and *Paris* near *London* in the learned vector space, capturing semantic relationships. This was achieved by training on large corpora to predict contexts of words. Word embeddings allowed algorithms to generalize better – understanding that if *movie* and *film* have similar vectors, a classifier trained on “great film” being positive could also recognize “great movie” as positive sentiment. Still, these embeddings handled words in isolation and did not solve understanding of whole sentences or context beyond word level.

### 2.3 Recurrent Neural Networks (RNNs) and LSTMs

Neural networks began to dominate NLP with Recurrent Neural Networks, which process sequences of words one at a time while maintaining a hidden state (a form of memory). An RNN can, in theory, retain information from earlier words to inform later ones. For example, in processing the sentence “The cake that I **baked** yesterday was delicious,” an RNN could carry the context of “cake” and “baked” when outputting a representation for “delicious,” helping a task like sentiment analysis or coreference resolution. However, **vanilla RNNs** struggled with long sentences due to vanishing/exploding gradient problems – they would “forget” earlier words as the sequence grew.

**LSTMs (Long Short-Term Memory networks)** and **GRUs (Gated Recurrent Units)**, introduced in the mid-2010s, tackled this by using gating mechanisms that better preserved long-term dependencies. LSTMs could handle sentences with dependencies across many words (e.g., understanding the subject of a sentence even after a lengthy clause). They powered early successes in machine translation and speech recognition. For instance, an LSTM-based translator could take an English sentence and output its French translation by reading the whole sentence (encoder) and then producing a translation (decoder) – a precursor to the sequence-to-sequence models used today.

## 2.4 Attention Mechanism – A Game Changer

Even with LSTMs, certain long sentences or paragraphs were hard to model. The **attention mechanism** (introduced around 2015) improved sequence models by allowing the network to *attend* to relevant parts of the input directly, rather than relying solely on the last hidden state. In translation, instead of an LSTM trying to encode an entire sentence into one final vector, an attention mechanism let the decoder look back at specific words of the input sentence when producing each word of the output. For example, when translating “The cat sat on the mat” to French, the model could attend to “cat” when producing “chat” and to “mat” when producing “tapis.”

The success of attention in improving translation and other tasks paved the way for the next leap – models that relied *only* on attention and did away with recurrence altogether.

## 2.5 Transformers: “Attention Is All You Need”

In 2017, the **Transformer architecture** was introduced <sup>2</sup>, famously described by its seminal paper “Attention Is All You Need”. Transformers use attention mechanisms to process entire sequences in parallel, rather than one step at a time as RNNs do. This change led to dramatic improvements in both speed (parallelization means training can utilize GPUs better) and capability (very long texts can be handled by attending to different parts directly).

A Transformer consists of an **encoder** (which reads the input text) and a **decoder** (which generates output, for tasks like translation or text generation). Key features of Transformers include:

- **Self-Attention:** Within the encoder, each word’s representation is updated by attending to every other word in the sentence. If we have a sentence like “The **python** slithered across the log,” the model can learn that “python” (a snake, in this context) is related to “slithered” more than to other words, by looking at all word pairs. Self-attention computes weights (scores) that indicate how important each surrounding word is to understanding the current word.
- **Multi-Headed Attention:** Instead of one attention process, Transformers use multiple attention “heads” in parallel. Each head can focus on different types of relationships. One head might focus on syntactic structure (e.g., subject-verb connections), another on semantic theme. They each produce an attention output, and the model combines them. This allows richer representations.
- **Positional Encoding:** Since Transformers don’t process words sequentially like RNNs, they need a way to know the order of words. Positional encodings (fixed or learned patterns added to word embeddings) give the model a sense of sequence. For example, a simple positional encoding might add a slight gradient to word representations that increases with position in the sentence, so the model knows word #5 comes after word #3.
- **Feed-Forward and Residual Layers:** After attention, there are fully connected layers applied to each position and residual connections (which help preserve information and stabilize training). The

Transformer's architecture with these components can be stacked into many layers (the original paper used 6 layers for encoder and decoder each), which allows building very deep models.

**Real-Life Analogy – Attention:** Think of a Transformer reading a paragraph as a student reading a textbook chapter. Instead of reading linearly word-by-word and trying to memorize everything (like an RNN), the student has the ability to flip back and forth (attend to relevant sections) instantly. When answering a question, the student can quickly look at the page where the relevant info is. This is akin to how attention allows a model to look at any part of the input when producing each piece of output, rather than only remembering from a one-pass read.

## Chapter 3: Birth of Large Language Models

With Transformers providing an effective architecture, researchers scaled them up, leading to what we now call Large Language Models. In this chapter, we discuss **what makes a model “large”** and highlight landmark LLMs.

### 3.1 What’s “Large” in LLMs?

“Large” usually refers to models with a huge number of parameters (often in the hundreds of millions to billions range) and trained on enormous text datasets. The size of a model is roughly proportional to its capacity to capture complex patterns. For example, **GPT-2** released by OpenAI in 2019 had ~1.5 billion parameters, which was considered astounding at the time, allowing it to generate surprisingly coherent text. Not long after, **GPT-3** in 2020 jumped to 175 billion parameters, demonstrating even more remarkable abilities like answering trivia questions, writing essays, or generating code from descriptions <sup>3</sup>.

But it’s not just parameter count – it’s also about training data and compute. GPT-3 was trained on hundreds of billions of words from the internet (books, articles, websites), learning grammar, facts, and some reasoning abilities from that broad coverage. **Large** implies both a high capacity (lots of parameters) and broad knowledge (extensive training data).

### 3.2 Landmark Models and Timeline

Let’s look at a brief timeline of influential language models and their contributions <sup>4</sup> <sup>3</sup>:

- **June 2018: GPT-1 (Generative Pre-trained Transformer)** – The first **GPT** by OpenAI showed that a Transformer-based language model pre-trained on unlabeled text can be fine-tuned for various NLP tasks to achieve strong results <sup>5</sup>. GPT-1 was relatively small (~117 million parameters) but introduced the paradigm of pre-training + fine-tuning for NLP.
- **October 2018: BERT (Bidirectional Encoder Representations from Transformers)** – Google’s BERT took a different approach by pre-training on a *masked language modeling* task (predicting missing words) and next sentence prediction <sup>5</sup>. BERT’s bidirectional nature (looking at both left and right context) made it excel at understanding text. It became the backbone of many NLP tasks (like Q&A, classification) but is not designed for text *generation*. BERT’s release, along with its open-source model, spurred a wave of research and model development.
- **February 2019: GPT-2** – An improved successor to GPT-1, with 1.5B parameters. GPT-2 generated such coherent and sometimes surprising text that OpenAI initially was cautious about fully releasing

it, citing concerns about misuse <sup>6</sup>. Eventually, the model was released and became a popular foundation model for text generation experiments.

- **October 2019: T5 (Text-To-Text Transfer Transformer)** – Google’s T5 treated every NLP problem as a text-to-text problem. It could take an input (like a question) and output a text (the answer). By unifying the format, the same model could be fine-tuned for translation, summarization, or classification (by outputting a label text) simply by prepending a task-specific prefix (e.g., “translate English to German: ...”). T5 showed the versatility of sequence-to-sequence Transformers on many tasks and came in sizes up to 11 billion parameters.
- **May 2020: GPT-3** – The leap to 175 billion parameters made GPT-3 famous even outside of AI circles <sup>3</sup>. Without any fine-tuning, it could be prompted to perform tasks (zero-shot or few-shot learning). For example, given just a few examples of a task in the prompt, GPT-3 could carry on and complete the task on new inputs. This emergent ability meant that for many applications, you could use GPT-3 via prompting alone, rather than collecting a task-specific dataset. It wasn’t perfect but was a massive step toward versatile AI.
- **Jan 2022: InstructGPT** – OpenAI fine-tuned GPT-3 using a technique called Reinforcement Learning from Human Feedback (RLHF) to make it follow user instructions better <sup>7</sup>. This model, and others like it, were more aligned to what users intend (e.g., if you ask for a recipe, it gives a straightforward recipe, not a random continuation about something else). InstructGPT was the basis for what became *ChatGPT*, which was released to the public in late 2022.
- **2023: LLaMA and Open-Source LLMs** – Meta (Facebook) released **LLaMA** in early 2023, a series of models (7B to 65B parameters) that achieved strong performance and were openly available to researchers <sup>8</sup>. This sparked a proliferation of fine-tuned variants by the community (e.g., Alpaca, Vicuna, etc., which are LLaMA models fine-tuned on instruction-following or conversational data). In mid-2023, **Llama 2** was released with a permissive license, allowing commercial use, and included a fine-tuned chat-oriented model out-of-the-box. Other notable open models include **BLOOM** (176B, a multi-language model from BigScience collaboration) and **Mistral 7B** (a compact model that surprisingly outperformed larger ones on certain benchmarks <sup>9</sup>). The open-source LLM movement is important for giving broader access to LLM capabilities without depending solely on API providers.
- **2023-2025: Multimodal and Specialized LLMs** – Latest developments have seen models like **GPT-4** which are *multimodal* (capable of processing text and images) and domain-specific LLMs such as **Med-PaLM** (Google’s medical LLM) or **Code LLMs** like OpenAI’s Codex and Meta’s CodeLlama, which are specialized in programming.

This timeline is not exhaustive but highlights how rapidly the field has evolved in just a few years. Each model built on prior innovations – and as hardware and data scale, we saw a virtuous cycle of bigger models leading to better performance on an ever-growing set of tasks.

**Real-Life Example – GPT-3’s Zero-Shot Magic:** To appreciate GPT-3’s leap, consider a task like writing a short poem about the sea. Before GPT-3, you’d likely need to train or fine-tune a model on poetry data to do this well. GPT-3, thanks to ingesting a huge swath of the internet (including poetry, literature), can do it without additional training. Simply prompt: “Write a four-line poem about the calm sea at sunrise.” GPT-3 will produce a poem that, while maybe not Shakespeare, is coherent and often surprisingly good. This ability to generalize to tasks with just a prompt was a new horizon for AI, showing how far a pre-trained LLM alone could go.

## Chapter 4: How LLMs Work Under the Hood

While the previous chapter listed the big names and timeline, this chapter dives a bit into **what happens inside an LLM** like GPT or BERT when it processes language. We'll keep it high-level and intuitive.

### 4.1 Training Objective – Predicting Words

At the core, an LLM is trained with a simple-sounding goal: *predict the next word (or token) in a sequence*. This is known as **language modeling**. During training, the model is given a huge number of text examples. For each example (which could be a sentence or paragraph), it hides a part of the text and tries to predict it. For a model like GPT (an autoregressive Transformer), it is trained on **causal language modeling** – given all the previous words, predict the next one <sup>10</sup>. For example, if the training text is “The cat sat on the”, the model might predict “mat” as the next word (because that phrase appeared often in training data). Initially, it guesses, then the training algorithm adjusts the model's internal weights to make that prediction more likely next time. Doing this billions of times makes the model progressively better at speaking “English” (or whatever language data it saw).

Models like BERT use a slightly different training: **masked language modeling**. They take a sentence like “The cat sat on the mat” and randomly mask out some words (e.g., “The cat sat on the [MASK]”). The model must predict the masked word (“mat”). BERT also had a next-sentence prediction task (predict if sentence B follows sentence A in the original text) to help it understand sentence relationships. The point is, through these training tasks, the model learns a statistically grounded understanding of language. It captures syntax (grammar rules) and a lot of world knowledge (because to predict a word, sometimes you need to know facts – if the text was “The capital of France is [MASK]”, the model must output “Paris”, meaning it has stored that fact).

**Why does this matter?** Because by the end of training, the LLM is not just a parrot memorizing text – it has generalized patterns. It has a notion that “Paris” often follows “capital of France”, that “once upon a” is usually followed by “time”, that “ice cream” is a noun phrase, etc. It compresses the knowledge from its training data into its parameters.

### 4.2 Inside a Transformer Block

An LLM typically has many *Transformer blocks* stacked on top of each other (like 12 layers, 24 layers, or more in large models). Let's peek into one block:

- **Self-Attention Layer:** This is where the model looks at the other words in the input. If our input sentence at some point is “Alice drove her car to the bookstore because she wanted to buy a novel.” When processing the word “she”, the attention mechanism allows the model to focus on “Alice” as the referent (among other words), because that's likely what “she” refers to. The attention produces for each word a weighted combination of all words, emphasizing related ones. Technical detail: each word's embedding is transformed into a Query, Key, and Value vector; attention scores are computed as Query dot Key (so basically how well one word's query matches another's key), those scores are normalized (softmax) to weights, and these weights multiply the Value vectors to produce a new representation for the word. Multi-head attention means this happens in parallel in different subspaces.

- **Feed-Forward Network:** After attention, for each position (word) we have an updated representation that has considered other words. This goes through a small two-layer neural network (applied independently to each position) which can mix information in new ways and add non-linearity (through activation functions like ReLU or GELU). This helps model complicated relationships and interactions that attention alone might not capture.
- **Residual Connections and Layer Norm:** To stabilize learning, the original input to a sublayer (like attention or feed-forward) is added to its output (this is residual addition). It's like saying "let the layer make refinements, but keep some of the original information for stability." After that, a normalization step ensures the scale of outputs is consistent.
- **Stacking Layers:** The output of one block becomes the input to the next. Each layer allows the model to develop more abstract features. Lower layers might focus on very local patterns (like word pair relations, or simple grammar), while higher layers might capture higher-level concepts (like the topic of a paragraph or an analogy in a sentence).

### 4.3 Prompting and Generation

When using an LLM for generation (like writing text), we **prompt** it with a context and let it generate the next word repeatedly. For example, if you prompt GPT-3 with "Once upon a time," it will sample a next word based on the probability distribution it has learned (likely "there" or "a" or similar as next word). Then it takes that whole sequence "Once upon a time there" and again predicts the next word, and so on. This iterative process continues until a stopping condition (like reaching a sentence end or a specified length). The "creativity" of the output can be adjusted by a parameter called *temperature* – higher temperature means more randomness (the model is more likely to pick a less probable continuation sometimes, which can lead to more surprising or creative text), whereas a low temperature makes it more deterministic and conservative.

**Real-Life Example – Autocomplete:** This process is exactly how modern email or messaging autocomplete works, albeit with smaller models. You start typing "I will be late to the" and it might suggest "meeting" next. That suggestion is effectively the model's next-word prediction. LLMs take this to the extreme: given a whole paragraph as a prompt, they can continue writing an entire essay. If you've used a phone keyboard that suggests the next word, you've interacted with a simplified language model in action.

### 4.4 Understanding versus Mimicry

A common question is: do LLMs "understand" what they write? The answer depends on what we mean by understand. LLMs do not *consciously* understand text like a human does – they don't have intent or true comprehension. They predict text based on patterns. However, the patterns they learn are so sophisticated that it often **appears** like understanding. For instance, if you ask an LLM a reasoning question, it may get the answer right by effectively mimicking the logical patterns it saw during training. If you ask "What color is the sun?" it will say "The sun is yellow" because that factual association was common in its training data. If you ask a more complicated question that requires multi-step reasoning, modern LLMs can perform *chain-of-thought* reasoning (especially if prompted to "think step by step"), which means they can generate intermediate steps that lead to an answer. They aren't guaranteed to be correct (they don't have a fact lookup unless augmented), but they often produce reasonable facsimiles of reasoning.

We should be aware that LLMs can also *hallucinate* – produce incorrect or nonsensical answers with great confidence – because if their training data had gaps or certain triggers, they might fill with the closest-

matching pattern which could be wrong. Later chapters on fine-tuning and alignment will discuss how to mitigate some of these issues by tailoring a model more closely to the desired behavior.

## Chapter 5: Using Pre-Trained LLMs Out-of-the-Box

Now that we know how LLMs work internally, let's discuss how to use them without any further training – essentially treating them as off-the-shelf tools. A **pre-trained LLM** can perform many tasks through prompting or simple wrappers. This chapter covers ways to leverage such models and the limitations of doing so.

### 5.1 Prompting for Tasks

A powerful aspect of large models like GPT-3 is that they can perform tasks just by providing a carefully crafted prompt (a technique known as **prompt engineering**). For example:

- **Text Generation:** If you want a short story about a cat, you can prompt the LLM: "Write a short story about a lonely cat who finds a new home." The model will continue with a story that fits the theme.
- **Question Answering:** Provide context followed by a question. E.g., prompt: "Context: Albert Einstein was a theoretical physicist who developed the theory of relativity. Question: Who developed the theory of relativity? Answer:" The model will likely answer "Albert Einstein."
- **Translation:** With models like GPT-3 or GPT-4, even if not explicitly trained for translation, you can do: "Translate the following English text to French: 'Good morning'." And it will output "Bonjour".
- **Summarization:** Prompt with an article text and say "Summarize the above article in one paragraph." The model attempts to condense the content.

This approach treats the model as a versatile tool. **Hugging Face** provides a high-level interface called **pipelines** in its Transformers library to use models for common tasks. For instance, using the Transformers Python library you could do:

```
from transformers import pipeline
# Load a pipeline for sentiment-analysis (this will automatically load a
# suitable pre-trained model under the hood)
classifier = pipeline("sentiment-analysis")

result = classifier("I love the new design of your website! It's so user-
friendly.")
print(result) # e.g., [{'label': 'POSITIVE', 'score': 0.9985}]
```

Similarly, there are pipelines for text generation, translation, question-answering, summarization, etc., which make using a pre-trained model one line of code. These pipelines abstract away the details of tokenization and model execution, returning human-readable results.



## 5.2 Using OpenAI API for Pre-trained Models

OpenAI provides API access to their models (like GPT-3, GPT-3.5, GPT-4). Without needing to know anything about model internals, you can get outputs via simple API calls. For example, using Python and OpenAI's library:

```
import openai
openai.api_key = "YOUR_API_KEY"

prompt = "Summarize the following:\n" + long_text
response = openai.Completion.create(
    model="text-davinci-003",
    prompt=prompt,
    max_tokens=100,
    temperature=0.7
)
summary = response['choices'][0]['text']
print(summary)
```

This will send the prompt to OpenAI's servers where the model (text-davinci-003, a variant of GPT-3) generates a continuation up to 100 tokens, and returns it. The advantage here is you don't need the compute power to run a 175B model yourself; the disadvantage is you rely on a paid service and must abide by their usage policies.

OpenAI's newer API (for the GPT-4 and GPT-3.5 "chat" models) uses a conversation format where you send a list of messages with roles (system, user, assistant) instead of a single text prompt. This more structured format is helpful for multi-turn dialogues and instructing the model on how to behave (via a system message like "You are an assistant that speaks like Shakespeare").

**Real-Life Example – Content Creation:** A marketing team can use a pre-trained LLM via API to generate product descriptions. They might not care how it works under the hood, only that if they provide a prompt like "Product: SuperShoe 3000. Features: lightweight, waterproof, self-lacing. Write a promotional product description.", the model will output a nicely written paragraph they can use as a starting draft. This is using the model out-of-the-box for generation. However, if the model sometimes uses a style or terms they don't like, they might consider fine-tuning it – which is exactly what later chapters will cover.

## 5.3 Limitations of Out-of-the-Box Models

While using pre-trained models directly is very powerful, you will encounter limitations:

- **Lack of Specificity:** The model's knowledge is broad but not specific to your domain or style. If you need a medical assistant bot, a general model might give incorrect medical advice or use layman language where you want clinical tone.
- **Inconsistency:** Without fine-tuning, you can't enforce certain behaviors. One day the model might answer a query verbosely, another day concisely, depending on subtle differences in prompt or

hidden randomness. This inconsistency can be frustrating if you need reliable formatting or tone (although you can mitigate it with carefully engineered prompts and few-shot examples).

- **Data Sensitivity:** Pre-trained models reflect their training data. They may be unaware of events after their training cutoff (for example, asking GPT-3, whose knowledge largely stops in 2021, about a late-2022 news event, it may not know or may guess wrongly).
- **Bias and Appropriateness:** Out-of-the-box models may produce biased or inappropriate content because they learned from the internet. Companies often put effort into filtering or moderating model outputs (OpenAI does this via content filters, etc.). If your use case requires a high degree of control (say, a model that talks to children, or one used in hiring decisions), simply using the raw model is risky without additional alignment steps.
- **Cost and Latency:** Using large models via API can be costly (OpenAI bills per token, so long documents or many requests rack up costs). Running them locally is also resource-intensive, possibly requiring GPUs. If you only need a smaller scope (like classifying tweets as happy or sad), a giant general model might be overkill in terms of compute.

These limitations hint at why **fine-tuning** is often beneficial. Fine-tuning can make a model more specific, consistent, and controlled for a given task or domain, as we'll start exploring soon.

## 5.4 When to Fine-Tune vs. When to Prompt

It's worth summarizing the decision of whether to invest in fine-tuning a model or try to solve your problem with prompting:

- If the task is straightforward and you have few examples, try prompting first. For instance, to get a summary of a single document, a prompt to GPT-4 might suffice without any new training.
- If the task requires the model to output in a very particular format or use specialized knowledge/jargon consistently, fine-tuning is likely worth it. For example, generating customer support emails in your company's style might work better if the model is fine-tuned on your past support email archive rather than hoping the right prompt always induces that style.
- If data privacy is a concern (you can't send data to an external API) or cost per query is high, fine-tuning an open-source model that you can run internally makes sense.
- If the model is almost doing what you want but making specific types of mistakes, fine-tuning on examples where it should do better can directly correct those mistakes, whereas prompt engineering might be an indirect and imperfect workaround.

In summary, prompting leverages the model's existing capabilities, while fine-tuning *molds* those capabilities to better fit a specific use case. Fine-tuning is the subject of the next parts of this book, starting from fundamentals and progressing to advanced techniques.

## Chapter 6: Fine-Tuning Fundamentals

Fine-tuning is the process of taking a pre-trained model and training it further on a specific dataset to adapt it to a particular task or domain <sup>11</sup>. It's like taking a broad education and giving it a specialization. This chapter lays out the fundamental concepts of fine-tuning, why it's done, and important considerations when fine-tuning LLMs.

## 6.1 Why Fine-Tune an LLM?

You might wonder, if LLMs are so powerful out-of-the-box, why do we need to fine-tune them at all? There are several compelling reasons:

- **Specialization:** Pre-trained LLMs are generalists. If you need a model to generate legal contract language or classify medical research papers by topic, fine-tuning on examples from that domain will yield much better performance. The model will learn domain-specific terminology and style. For instance, GPT-3 might not naturally produce perfectly formatted legal clauses, but if fine-tuned on a corpus of contracts, it can learn to do so.
- **Performance Boost:** On specific tasks (especially those with a well-defined correct output), a fine-tuned model can far outperform a generic model. A famous example: GPT-3 can do basic arithmetic or reasoning with prompts, but fine-tuned models like **GPT-3.5** (via RLHF, which is a kind of fine-tuning) or smaller fine-tuned models can follow instructions more reliably and even match or beat a larger model that isn't specialized <sup>12</sup>. In some internal tests, a fine-tuned GPT-3.5 on a narrow task outperformed GPT-4 on that task <sup>12</sup> – illustrating that size isn't everything; being specialized matters.
- **Cost Efficiency:** A fine-tuned smaller model can sometimes replace a larger model for a specific task. If you have a high-throughput application (say, classifying thousands of social media posts per minute for content moderation), you might fine-tune a 1B-parameter model to do it with 95% accuracy, instead of calling a 175B-parameter model via API for each post. The smaller model could be deployed on your own servers at lower cost.
- **Alignment and Control:** Fine-tuning can instill desired behaviors. For example, OpenAI fine-tuned base GPT-3 with human feedback (InstructGPT) to refuse inappropriate requests and follow user instructions better. By fine-tuning, you can make a model more polite, or always respond in rhyme, or never mention certain banned phrases, depending on your requirements (assuming you have or can create training data that reflects these behaviors).
- **Using Proprietary Data:** Your use case might involve private data – say, internal company documents for a Q&A bot. Rather than feeding those documents to an external model every time (which might not even be allowed), you can fine-tune a model on that data so it “absorbs” the knowledge. Then, queries can be answered by the model directly. (However, be careful: fine-tuning on sensitive data means the model weights now contain that data in some form. We will touch on privacy considerations later.)

## 6.2 How Fine-Tuning Works

On a technical level, fine-tuning is simply **continuing the training** of the model on new data, usually with a **task-specific objective**:

- For a **classification task**, you'll have labeled examples (e.g., movie reviews labeled positive/negative). You typically attach a simple classifier layer on top of the model (if not already present) and train the model to minimize classification error on those examples.
- For a **text generation or sequence-to-sequence task**, you fine-tune the model to maximize the probability of the target text given the input. For example, for summarization, the training data might be pairs of (article, summary). You fine-tune a model like T5 or BART on these pairs, so it learns to produce the summary from the article. For an autoregressive model like GPT-2, if you want to fine-tune it to generate Shakespearean text, you might just feed it lots of Shakespeare and it will adjust its weights to better mimic that style.

Fine-tuning uses the same principles as training: you define a loss function (error measure) on your new dataset and run backpropagation to adjust the model's weights and minimize that loss. The key difference is **starting from a pre-trained state** instead of random initialization. Starting from a pre-trained model means: (a) You need **much less data** to get good performance (often a few hundred or thousand examples can suffice for an adaptation, whereas training from scratch might need millions); (b) The model will train **faster** (converge in a few epochs) because it already “knows” a lot and just needs to adjust slightly; (c) You typically use a **lower learning rate** for fine-tuning than you would for training from scratch, to avoid clobbering the pre-trained knowledge.

Think of it as teaching a skilled translator a new language: since they already know multiple languages, you can explain the new one by analogy and differences, rather than starting with “this is a noun, this is a verb.” The training is a refinement process.

It's important to **hold out some data** as a validation set during fine-tuning to monitor performance and avoid overfitting. Overfitting can happen if your fine-tune dataset is very small – the model might become too tailored to those examples and perform worse on new examples. Techniques like early stopping (stop training when validation performance stops improving) are used.

### 6.3 Fine-Tuning vs. Prompt Engineering

There's sometimes a grey area between prompting and fine-tuning. For instance, one can “few-shot train” a model by putting some examples in the prompt, effectively conditioning the model to perform a task. This is not exactly updating model weights, but it's giving it a demonstration of the task each time. Few-shot prompting can be powerful (and doesn't require any model changes), but it has downsides: you might hit context length limits (many models can only take a few thousand tokens of input), and it's not persistent (you have to supply those examples every time). Fine-tuning is like permanently baking those examples into the model weights. After fine-tuning, you can query the model with just the task input, not the examples, which is more efficient.

**Hybrid approaches** also exist: for example, prompt the model to get some initial results, and then fine-tune on those results along with corrections. But typically, you'll either decide to rely on prompting (if results are good enough and data is scarce or constantly changing) or invest in fine-tuning (if you have stable data and need better performance).

### 6.4 Considerations and Pitfalls

- **Catastrophic Forgetting:** If the new fine-tune dataset is small or very different from the original training data, the model might partially forget things it knew before. For example, if you fine-tune a general model on a narrow dataset of legal documents only, it might become great at legal language but could degrade on everyday queries or other tasks it once did well. To mitigate this, sometimes a small learning rate is used, or one might mix some of the original data (if available) to retain generality (this is advanced and known as **continual learning** issues).
- **Overfitting:** As mentioned, if you only have, say, 100 examples and you fine-tune a huge model on them, it might just memorize them (especially if run for too many epochs). The validation set helps detect this (training loss keeps going down, but validation loss starts going up – a classic sign of overfitting). In such cases, techniques like data augmentation or regularization might help, or you might opt for few-shot prompting instead if fine-tuning isn't yielding generalizable results.

- **Evaluation:** It's crucial to evaluate a fine-tuned model on appropriate metrics for your task. For classification, accuracy, precision/recall, F1 might be used. For generation tasks like summarization, metrics like **ROUGE** or **BLEU** compare the model output to reference outputs. However, for generative tasks, human evaluation is often needed to truly assess quality (because automatic metrics only go so far – a model might generate a good summary that doesn't match the reference exactly, and it would score low on ROUGE despite being fine). We'll discuss evaluation in each task chapter.
- **Model and Data Compatibility:** Not every model can be fine-tuned on every task directly. For instance, BERT is not a text generator; it's good for understanding tasks (classification, extraction). GPT-2 or GPT-3 style models are good for generation but less straightforward for classification (though they can, with prompting). If you have a sequence-to-sequence task like translation or summarization, you'd typically use an encoder-decoder model (like T5, BART) for fine-tuning, not a plain decoder-only model – because the architecture fits the task better (encoder to read input, decoder to generate output). You could prompt a GPT model to do summarization, but if fine-tuning, using a seq2seq model might be easier with parallel data. So, picking the right pre-trained model to fine-tune is a key step (and we'll cover in the next chapter how to choose models).

**Real-Life Example – Fine-Tuning Improves Answers:** A startup wants to build a support chatbot for their software product. The general GPT-3 can answer some questions correctly via prompting, but often it hallucinates features or gives overly verbose answers. The startup gathers a small dataset of real support Q&A pairs and fine-tunes GPT-3 (or a smaller open model) on this dataset. The fine-tuned model now answers with concise, product-specific information and makes far fewer errors on typical queries. Essentially, it has *learned* the support knowledge base. It might even use the exact phrasing the company prefers (since that was in the training examples). This is the power of fine-tuning – aligning the model's outputs with the needs of a particular application.

## Chapter 7: Preparing Data for Fine-Tuning

“Garbage in, garbage out” – this adage is especially true when fine-tuning LLMs. The quality and format of your fine-tuning data largely determine how effective the fine-tuned model will be. This chapter covers **data preparation**: how to format and preprocess your dataset for various tasks, with real examples.

### 7.1 General Data Collection Tips

- **Gather Relevant Data:** Identify what kind of data will teach the model the desired behavior. For a sentiment classifier, you need texts labeled with sentiments. For a custom chatbot, you need example dialogues or question-answer pairs in the style you want. If you don't have data, you might have to create or synthesize it (for instance, writing a few hundred Q&A pairs that cover your product's FAQs).
- **Quality Over Quantity (to a point):** It's better to have a smaller set of clean, representative data than a huge set of noisy data. Remove or fix glaring errors in the data, as the model will try to learn from everything. That said, deep learning models love quantity, so if you can get both quality and quantity, that's ideal. But don't sacrifice quality thinking the model will just “figure it out” – biases or errors in training data often directly translate to the model's output.
- **Format Consistency:** The format of the input and output in your training examples should be consistent. If you are doing a dialogue model, for instance, you might decide that each training example will look like:

**Input:** "User: <user question>\nAssistant: <assistant answer>" (and the model's job is to output the assistant answer when given the user question).

Or you might separate roles with special tokens or IDs in a structured format. The key is to use the same structure every time, so the model picks up on it.

- **Train/Validation Split:** Always set aside some portion of data for validation (and possibly also for a test set if you plan to measure final performance). A typical split might be 80% training, 10% validation, 10% test. The validation is used during training for checking performance and deciding hyperparameters, the test is used after training to report how well the model does on unseen examples. If your dataset is very small (hundreds of examples), you might use cross-validation or just a simple split like 90/10. But have *something* to test on that wasn't used in training.

## 7.2 Data for Text Classification

**Format:** Usually a table or CSV where each sample has a text and a label. For example:

| text                                                        | label    |
|-------------------------------------------------------------|----------|
| "I absolutely loved the new Batman movie! Best one so far." | positive |
| "The restaurant was dirty and the food was bland."          | negative |
| "It's an okay phone. Does the job, nothing fancy."          | neutral  |

If using Hugging Face's `datasets` library or PyTorch dataloader, you'll convert these to something like `{"text": "I absolutely loved ...", "label": 2}` (if you map positive=2, neutral=1, negative=0, for instance). The label might be numeric internally, but you keep track of the mapping to human-readable form for output.

### Preprocessing:

- Tokenization is needed (the model's tokenizer will convert text to token IDs). Usually, you'll do this in a pipeline that feeds the model. You might also truncate texts to a maximum length and possibly pad shorter ones to a fixed length (especially if using batches, to have a uniform input size). For example, many BERT models have a max length of 512 tokens. If some reviews are longer, you either truncate or split them. - If labels are textual originally (like "positive", "negative"), convert them to integers for training, since classification models typically output a probability for each class index.

**Real Example:** The **Yelp Reviews** dataset is a common example. It has texts and a star rating 1-5. If we wanted to fine-tune a classification model to detect sentiment (positive/negative), we could map 1-2 stars to negative, 3 neutral, 4-5 positive, and label accordingly. In the Hugging Face Transformers docs, an example shows loading Yelp and fine-tuning a BERT model on it <sup>13</sup> <sup>14</sup>. They tokenize the text and use `AutoModelForSequenceClassification` with `num_labels=5` for the 5-star rating classification <sup>14</sup>. The dataset might be large, so for experimentation, they even suggest using a subset first <sup>15</sup> (1,000 examples) to iterate quickly, then scale up. This is a good practice: do a small test run to ensure your pipeline works end-to-end, then run on full data.

## 7.3 Data for Text Generation

This includes tasks like *open-ended generation* (creative writing, dialogue without a fixed correct answer) or *structured generation* (like generating an output given an input, e.g., style transfer). The preparation depends on whether your task has an explicit input-output pair or it's just modeling a style.

**Unconditional Generation (Style Finetuning):** Suppose you want to fine-tune GPT-2 to write in Shakespearean style. You might gather Shakespeare's plays or sonnets as training data – basically a large corpus of text in that style. Then you fine-tune the model on that text with a language modeling objective (predict next token). Here, you don't have an "input" other than the model's own generated history during training – you're simply training it to continue text in that style. The data preparation means cleaning the text, maybe adding markers between works if needed, and possibly breaking long texts into manageable chunks (so you feed in e.g. sequences of, say, 256 or 512 tokens from Shakespeare at a time). The model will learn to mimic the style, diction, and content of Shakespeare. Later, at generation time, you can prompt it with a short phrase and it will continue in that style.

**Prompted Generation (Prompt-Completion Pairs):** In other cases, you have input-output pairs. For example, custom chatbot responses (where the input is a user query and output is the desired answer), or code generation (input is a docstring, output is the code), or **creative tasks** like "Continue this story" (input is story start, output is continuation). For these, your dataset would be a collection of prompts and corresponding desired outputs. A single example might look like:

`{"prompt": "<prompt text>", "completion": "<output text>"}` for OpenAI style JSONL format, or for training a causal model, you might join them with a separator token. In OpenAI's fine-tuning (for older models like davinci), they used prompt-completion pairs in JSONL where each line has `{"prompt": "text", "completion": "text"}`.

**OpenAI Chat Format:** If you plan to fine-tune using OpenAI's API for models like GPT-3.5 Turbo, the data is formatted a bit differently – each example is a conversation. According to OpenAI's docs, each training example is a JSON with a **messages** list, containing a sequence of role/content pairs <sup>16</sup>. For example:

```
{ "messages": [
  { "role": "user", "content": "How do I reset my password?" },
  { "role": "assistant", "content": "To reset your password, go to the login
page and click 'Forgot password'..." }
]
```

This would teach the assistant how to respond to that user question. You can include a system role too if needed for instructions. When preparing such a JSONL, you must ensure it's properly formatted (valid JSON with double quotes, etc., as the OpenAI API expects) <sup>17</sup> <sup>18</sup>. After preparing the file, you'd upload it to OpenAI and start the fine-tune (we will detail that process in the Fine-Tuning with OpenAI chapter).

**Dataset Size:** For generation tasks, more data helps capture style diversity. If fine-tuning for a chatbot, you might have at least a few thousand Q&A pairs to make it robust. But even a few hundred carefully written pairs can give a noticeable personalization to the model.

**Cleaning:** Make sure to remove any unwanted text in outputs (like system messages, artifacts, or offensive content) because the model will faithfully reproduce patterns. If your dataset is scraped dialogues from forums, you might want to filter out profanity or specific user mentions depending on your desired tone.

## 7.4 Data for Dialogue (Chatbot) Fine-Tuning

Dialogue fine-tuning is a special case of prompted generation, but worth discussing separately due to the format. If you are fine-tuning an open-source model (not via OpenAI API) to act as a chatbot, one common approach is to create a single long sequence per conversation example that includes the back-and-forth. For instance:

**Example conversation for training:**

```
<|start_of_text|>
User: Hello, how are you?
Assistant: Hi there! I'm doing well, thank you. How can I assist you today?
User: I'm looking for a book recommendation.
Assistant: Sure! I'd be happy to help. What genres do you enjoy?
<|end_of_text|>
```

During training, you would feed this entire block as one sequence to a causal language model (like GPT-2 or LLaMA). The model sees the pattern that after “User:” comes user query text, after “Assistant:” comes the assistant response. Over many such examples, it learns to take a user prompt and generate the assistant continuation. There might be special tokens or separators to clearly mark user vs assistant. In the example above, I just used plain prefixes, but one could use actual special tokens if the model’s tokenizer is configured with them.

Some fine-tuning datasets (like the Stanford **Alpaca** dataset, which was based on the self-instruct technique) use an instruction-response format. For example:

```
Below is an instruction and a response.
### Instruction:
Write a short poem about AI.
### Response:
Sure. Here is a poem about AI:
AI in the morning light,
Thinking deep and getting it right, ...
```

This format was used to fine-tune a model to follow instructions better. The exact format isn’t universal – the key is consistency and including whatever cues you want the model to pick up. If you want your chatbot to respond cheerfully and with markdown formatting for certain outputs, include examples of that.

**Note on Multi-Turn:** You can train multi-turn dialogues or single-turn QA pairs. Multi-turn (as in the example) gives the model a sense of context continuation, but if you mostly care about single Q -> A mapping, you can keep each example just one question and one answer to simplify.



**Real Example:** The **OpenAI ChatGPT** was trained via a mix of methods including supervised fine-tuning on demonstrations. For a simpler scenario, the **PersonaChat** dataset has conversations between a user and an assistant with personas. If fine-tuning a chatbot, one could use such conversation data. The persona information can be injected as a system prompt or initial message. Another example: the **Wizard of Wikipedia** dataset where the assistant provides knowledgeable responses with evidence – if you fine-tune on that, your bot can learn to cite facts (though it might cite from hallucinated sources unless grounded; retrieval augmentation is another topic beyond just fine-tuning).

The bottom line: structure your dialogue training data to reflect how you want the conversation to flow. The model will learn the pattern you show. If you never include long user messages in training, it might struggle when a user sends a long paragraph; try to include a variety of scenarios.

## 7.5 Data for Summarization and Translation

These are **sequence-to-sequence tasks** – mapping an input text to an output text (summary or translation). The data will be pairs of input-output.

For summarization, datasets like **CNN/DailyMail** have articles and their summaries (written by journalists). Each example is fairly long input (news article) and a paragraph summary. For translation, say English-French, you have a sentence or paragraph in English and its French equivalent.

**Format:** If using an encoder-decoder model (T5, BART, etc.), you typically feed the input to the encoder and train the decoder to produce the output. With Hugging Face's `Seq2SeqTrainer` or similar, you might prepare a dataset such that each entry has `{"input_text": "...", "target_text": "..."}` . You then use a tokenizer to encode both, where the model is set up with a tokenizer that can handle the two sides appropriately.

T5, for example, expects a prefix on the input to indicate the task <sup>19</sup>. Google's T5 uses prompts like `"summarize: <article text>"` as the model input for summarization tasks <sup>19</sup>. So your preprocessing will prepend "summarize: " to every input text <sup>19</sup>. This cues the model (since it was pre-trained on multiple tasks) to perform summarization. If you were doing translation with T5, you might prepend "translate English to German: " to the English sentence as the input. If using BART or other models not requiring an explicit prefix, you might not need such a prefix, but it can still be helpful to mark the task.

**Truncation:** Summarization data often has long inputs. Be mindful of the model's max input length (e.g., 1024 tokens for BART). If an article is longer, some strategies: truncate the article (possibly losing info), or split into chunks and summarize each chunk, or use a model that can handle long input (some newer LLMs allow 4k, 8k, or more tokens). For initial fine-tuning experiments, many just truncate to a limit.

**Metrics:** It's useful to compute metrics like ROUGE during or after training to see how close the generated summary is to the reference. Libraries like Hugging Face's `evaluate` provide a ROUGE metric out of the box <sup>20</sup> <sup>21</sup>. In training, a `compute_metrics` function can decode the model's output and compare to the target text to give you an average ROUGE score <sup>22</sup>. This helps in development – for example, if fine-tuning for too long leads to the model copying large parts of the input (thus increasing ROUGE scores artificially, which can happen if the model learns to just select sentences rather than paraphrase), you'll catch that by inspecting outputs.

**Data Example:** Let's say we prepare a small summarization dataset of movie plot summaries. An entry might be: - **Input:** (Plot of a movie) "A group of friends go on an adventure... [several paragraphs]." - **Target Summary:** "A group of friends embark on a quest, facing challenges, and ultimately find the treasure they were seeking."

We'd feed the input to the model and expect it to produce something close to the target summary. Because summarization is an inherently open-ended task (there are many ways to summarize), the model won't output exactly the reference, but we train it to capture the key points.

## 7.6 Tools for Dataset Preparation

A few useful tools and libraries: - **Hugging Face Datasets:** `datasets` library can load many common datasets with one line (e.g., `load_dataset('imdb')` for movie reviews, `load_dataset('cnn_dailymail', '3.0.0')` for summarization, etc.). It also helps with splitting and mapping (tokenization) <sup>23</sup> <sup>24</sup>. As seen, we can split a dataset into train/test easily <sup>25</sup>. It handles caching and is memory efficient. - **Pandas:** For CSV data, pandas is great to inspect and clean. You can then convert to a list of dicts or directly to a `datasets.Dataset` using `Dataset.from_pandas()`. - **OpenAI Tools:** If fine-tuning using OpenAI's API, they have a CLI tool to prepare data. For example, `openai tools fine_tunes.prepare_data -f <LOCAL_FILE>` which will suggest changes or verify format for you. This is handy to catch formatting issues before launching a fine-tune job. - **Data Augmentation:** In some tasks, you might have techniques to augment data. For classification, you can rephrase sentences using an LLM (careful not to change label meaning). For summarization, maybe use multiple reference summaries if available (or back-translate summaries to get paraphrases). Augmentation can effectively increase data size, but always review augmented data quality.

**Real-Life Example - Preparing a Custom Dataset:** Suppose we want to fine-tune an LLM to act as a programming help chatbot (like StackOverflow Q&A style). We might scrape a StackOverflow dump for Q&A pairs on programming. Then: - Filter to a certain domain (e.g., only Python-related questions if we want a Python expert bot). - Remove any questions with no accepted answer (to ensure quality answers). - The input could be the question title + description, and the output the accepted answer. - We might also prepend something like "Question: ... Answer: ..." to each, so the format is consistent. - Split into train/val. Perhaps keep 10% of Q&A pairs as validation to see if the model is learning to provide correct answers. - During fine-tuning, the model hopefully learns to produce answers like those on StackOverflow (which often include code snippets, technical details, etc.). After fine-tuning, we test it on some known question and see if it gives a good answer. We might compare with the original answer or just evaluate qualitatively.

With data ready, we can proceed to actual fine-tuning. But before we jump in, let's cover the tools and platforms that make fine-tuning feasible, especially focusing on Hugging Face and OpenAI since we will use those.

## Chapter 8: Tools and Platforms for Fine-Tuning LLMs

Fine-tuning a large model can be resource-intensive. Luckily, there are tools that simplify the coding and platforms that provide the needed compute. In this chapter, we discuss the practical aspects: what software libraries to use, what kind of hardware is needed, and how to leverage cloud or other services.

## 8.1 Hardware Requirements

Fine-tuning large models **requires a GPU (or several)** in most cases, because CPUs are far too slow for this kind of neural network training. The exact GPU memory needed depends on model size and batch size: - Smaller models like `distilBERT` or `GPT-2 small` (117M parameters) can be fine-tuned on a single consumer-grade GPU (like 8-12 GB memory) comfortably. - Medium models like `BERT-large` (~340M params) or `GPT-2 XL` (1.5B) might require 12-24 GB, especially for larger batch sizes. A 16GB GPU can do a lot, maybe with batch size 8 or so for 1.5B model. - Truly large models (like 6B, 7B, 13B, or beyond) typically need either very high memory GPUs (40GB A100s are often used for 13B+) or one can use tricks like model parallelism or 8-bit quantization to squeeze them onto smaller GPUs. Alternatively, one can use multiple GPUs to split the model. - Cloud services (AWS, GCP, Azure) and specialized providers (Paperspace, Lambda Labs, etc.) let you rent A100s or V100s for hourly rates. Google Colab and Kaggle Kernels sometimes provide free T4 (16GB) or even occasional A100 (40GB) in their pro versions, which can be enough for moderate fine-tunes.

**Storage:** Also note, the model checkpoints can be a few GB in size, and when fine-tuning, if you save checkpoints each epoch, you'll need disk space. And data loading should be reasonably fast (an SSD is good; if reading from slow network or drive, it could bottleneck training).

## 8.2 Hugging Face Transformers and Trainer

One of the most popular libraries for fine-tuning is **Hugging Face Transformers**. It abstracts a lot of boilerplate. The key classes and functions: - `AutoTokenizer` and `AutoModelForX`: These allow quick loading of pre-trained models and their tokenizers for a given task. For example, `AutoModelForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=3)` gives a BERT ready for a 3-class classification. - `Trainer` and `TrainingArguments`: The `Trainer` API <sup>26</sup> <sup>27</sup> simplifies the training loop. You provide the model, training data, an optimizer schedule (through training arguments), and it handles the rest (forward pass, loss calc, backward pass, optimizer step, evaluation, etc.) <sup>28</sup>. It also integrates with the Hugging Face Hub if you want to upload models, and can log metrics to TensorBoard or Weights & Biases easily. - Data Collators: These are helper classes to batch data and pad sequences properly. For example, `DataCollatorWithPadding` can dynamically pad sequences in a batch to the longest sequence length in that batch (saves padding computation compared to padding everything to max length).

### Basic Fine-Tune Loop with Trainer (Pseudo-code):

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM, Trainer,
TrainingArguments

# Load model and tokenizer
model_name = "t5-small"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

# Prepare dataset (assuming a HuggingFace Dataset object with 'input' and
'output' fields)
```

```

def preprocess(example):
    # Add T5 task prefix and tokenize
    inputs = ["summarize: " + doc for doc in example["input"]]
    model_inputs = tokenizer(inputs, max_length=512, truncation=True)
    # Tokenize targets with text_target to get labels
    labels = tokenizer(text_target=example["output"], max_length=128,
truncation=True)
    model_inputs["labels"] = labels["input_ids"]
    return model_inputs

tokenized_dataset = raw_dataset.map(preprocess, batched=True)

# Define training arguments
args = TrainingArguments(
    output_dir="t5_finetune",
    per_device_train_batch_size=4,
    per_device_eval_batch_size=4,
    num_train_epochs=3,
    learning_rate=5e-5,
    evaluation_strategy="epoch",
    save_strategy="epoch",
    logging_steps=50,
    logging_dir="logs"
)

# Instantiate Trainer
trainer = Trainer(
    model=model,
    args=args,
    train_dataset=tokenized_dataset["train"],
    eval_dataset=tokenized_dataset["validation"],
    tokenizer=tokenizer,
    compute_metrics=compute_metrics_function # optional, for eval
)

trainer.train()

```

This is a generic blueprint. The Trainer will handle the loops. The `compute_metrics_function` can be defined to compute say accuracy for classification or ROUGE for summarization as needed.

Hugging Face provides **examples** scripts (in their Transformers GitHub repo) for common tasks which basically do the above. For instance, `run_glue.py` for classification on GLUE tasks, `run_summarization.py` for summarization, etc., which one can use out-of-the-box by specifying model and data.

**Why use Trainer vs. custom training code?** Trainer is high-level and covers 90% of use cases, including advanced things like distributed training, mixed precision (fp16) training to speed up and save memory, gradient accumulation, etc. However, there may be times you want more control – then you'd write your own loop with PyTorch. But for learning and for most projects, Trainer is a huge time saver.

### 8.3 Parameter-Efficient Fine-Tuning Tools

If you lack the GPU resources to fine-tune all 175B parameters of a model, don't worry – there are techniques and libraries to help: - **Hugging Face PEFT (Parameter-Efficient Fine-Tuning)** library provides implementations of methods like **LoRA**, **Prefix Tuning**, **Adapters**. These allow you to train only a small fraction of parameters (like <1%) and still adapt a big model. The rest of the model can even be kept in half precision or 8-bit to save memory (see **QLoRA**, which fine-tunes in 4-bit precision <sup>29</sup> <sup>30</sup>). - With PEFT, you can do something like:

```
from peft import LoraConfig, get_peft_model
config = LoraConfig(r=8, lora_alpha=32, target_modules=["q_proj", "v_proj"],
bias="none", task_type="CAUSAL_LM")
model = get_peft_model(model, config)
```

And then train as usual. This injects trainable low-rank matrices into the model's attention layers but freezes the original weights. It significantly reduces memory usage. For example, LoRA has been shown to fine-tune a 7B model on a single GPU by only training maybe ~30 million parameters (the LoRA params) <sup>31</sup> <sup>32</sup>. - **Bit Quantization:** Libraries like **bitsandbytes** allow loading models in 8-bit or 4-bit precision. QLoRA (Quantized LoRA) approach loads the model in 4-bit and then applies LoRA on top <sup>29</sup>, enabling fine-tuning of 65B models on a single high-end GPU (like 48GB) which was previously impossible <sup>33</sup>. Tools to note: `BitsAndBytesConfig` in HF Transformers (used in [19] snippet to load a model in 4-bit). - If doing research or complex fine-tune like RLHF, you might need **TRL (Transformer Reinforcement Learning)** or custom code, but that's beyond simple fine-tuning.

For our book's practical chapters (generation, classification, etc.), we'll largely assume classical fine-tuning or mention these techniques when relevant, but it's good to know these exist if you find standard fine-tuning doesn't fit in your environment.

### 8.4 OpenAI API for Fine-Tuning

OpenAI offers fine-tuning for certain models via their API. The flow is: 1. Prepare a dataset in JSONL with prompt-completion pairs (for their older models like Ada, Babbage, Curie, Davinci) or messages format for GPT-3.5 Turbo. 2. Upload the file with `openai.File.create(...)` <sup>34</sup> – the file is stored and given an ID on their server. Ensure it's processed (status "processed") <sup>35</sup>. 3. Create a fine-tuning job by specifying base model, training file ID, (optional validation file ID), and some hyperparameters like number of epochs (OpenAI might have defaults) <sup>36</sup>. 4. The job runs on OpenAI's side (you can check status or wait for completion). If successful, you get a model ID for the newly fine-tuned model <sup>37</sup>. 5. You can then use `openai.Completion.create(model="new-model-id", prompt="...")` to use it (or the chat equivalent if it's a chat model).

Some specifics: - OpenAI as of 2023 allowed fine-tuning for `davinci`, `curie`, etc. and recently for `gpt-3.5-turbo`. By 2025, presumably GPT-4 might be allowed in some form, but let's focus on the available info: the process is similar but with the new messages format for chat models. As seen, for GPT-3.5, each training example is a list of messages with roles <sup>16</sup>. - They also allow a `suffix` parameter which appends a custom name to your fine-tuned model for easier reference. - Pricing: OpenAI charges per token for training and usage. Fine-tuning GPT-3.5 might cost a certain amount per 1K tokens (for training, maybe a few cents per 1K tokens processed, and similarly for inference). They had pricing like \$0.03 per 1K tokens for training GPT-3.5 as of 2023, but check current rates. This is an important consideration – fine-tuning a very large dataset could be pricey. - One limitation: OpenAI's fine-tuned models still have to obey the base model's capacity and restrictions. For example, if base GPT-3 had a certain level of knowledge cutoff or certain banned content rules, the fine-tuned model doesn't remove those (in fact, OpenAI may review fine-tuning data and reject if it's against their policies). Also, you can't change fundamental model architecture – e.g., you can't make GPT-3 suddenly output 10,000 token stories if the limit was, say, 2048 tokens.

We will have a dedicated chapter on using OpenAI's fine-tuning, but here the point is that using OpenAI is more "data prep and API calls" rather than coding a training loop. It's simpler in that you don't worry about GPUs or training code, but you trade off control and have to pay for usage.

## 8.5 Monitoring and Logging

During any fine-tuning, you'll want to monitor the process: - **Logging:** The `TrainingArguments` in HF can log training loss every few steps, and evaluation metrics each epoch. These can be printed to console or saved to a log file. For more sophisticated logging, you can integrate WandB (Weights & Biases) by just setting `report_to="wandb"` in `TrainingArguments` (and logging in to WandB). This will give a nice dashboard of loss curves, etc. - **Checkpoints:** Save checkpoints at least at the end or every epoch (Trainer does this if `save_strategy="epoch"`). If training crashes or if you want to pick the best epoch model (not necessarily last epoch if it overfits), having checkpoints is useful. - **Validation Performance:** Keep an eye if validation loss starts increasing while training loss still decreases – that's overfitting. You might stop early if that happens or use early stopping callback. Alternatively, if after first epoch you see hardly any improvement or too slow, you might adjust hyperparams (like learning rate). - **Hyperparameter Tuning:** Don't be afraid to try different learning rates, batch sizes. A common range for learning rate in fine-tuning is  $1e-5$  to  $5e-4$  for Transformers. If the model is not converging, maybe LR is too low; if it's diverging (loss going NaN or huge), LR might be too high. - **Gradient Accumulation:** If you can't fit a batch of, say, 16 examples due to memory, you can accumulate gradients. E.g., set batch size to 4 and `gradient_accumulation_steps=4` to effectively have a 16 batch (the trainer will do four forward passes, sum grads, then do one optimizer step). This helps use smaller GPUs for larger effective batch.

## 8.6 Ethics and Safety Measures in Fine-Tuning Tools

Given we mention alignment, a note: If your fine-tuning might result in a model that can produce sensitive or unsafe outputs, you should include that in data design (for example, include a lot of "If user asks for X, respond with refusal" examples if you want it safer). OpenAI's own fine-tuning guidelines forbid certain content in the training data, and they will check it. For open models, it's up to you to ensure compliance if deploying – consider fine-tuning with a dataset that has been scrubbed for extreme content if it's likely an issue.

**Real-Life Example – Using Colab for Fine-Tuning:** Let's say you don't have a local GPU, but you have Google Colab. Colab often gives a Tesla T4 (16GB) or Tesla P100 (16GB) which is enough for many fine-tunes. You can: - Upload your dataset to Colab (or better, store in Google Drive or somewhere accessible). - Install Transformers (`!pip install transformers datasets accelerate`) in the Colab. - Use the Trainer as shown. Perhaps fine-tune a `distilGPT-2` (small GPT-2) on your text data. With 16GB, you could even do GPT-2 medium for short texts. - Monitor the cell output (it'll show training loss and eval metrics if set). - Save the model to your Google Drive or download it after training.

Alternatively, Colab Pro sometimes offers V100 or even an A100, which can handle bigger models or faster training.

After covering tools and set-up, we're now ready to get hands-on with specific fine-tuning scenarios: custom text generation, classification, chatbot, and summarization, each in its own chapter with practical guidance and examples.

## Chapter 9: Fine-Tuning for Custom Text Generation

In this chapter, we walk through fine-tuning an LLM for *custom text generation*. This could mean adapting a model to generate text in a specific style, genre, or following specific patterns. We'll use Hugging Face Transformers for an example, and also mention how one might do generation fine-tuning with OpenAI API.

### 9.1 Defining the Task – Custom Text Generation

"Text generation" is a broad term. To narrow it down, let's pick a concrete use-case: **fine-tuning a model to generate customer service responses** in a specific style. Imagine we have a company that wants an AI to draft responses to customer emails. They have a lot of example Q&A from their support team that they can use as training data. We'll consider that our task – given a customer query, generate a helpful, polite answer following the company's style guidelines. This is essentially a conditional generation task (input = customer email, output = support response).

Alternatively, one could fine-tune for unconditional generation (like writing poetry). The process is similar, except without a distinct input-output pair – you'd train the model on the text you want it to emulate, possibly with some prompt context to start it off. But our example with Q&A covers the more general case.

### 9.2 Choosing a Base Model

For text generation, a **decoder-only model** (like GPT-2, GPT-Neo, GPT-J, etc.) or a **seq2seq model** (like T5, BART) can be used. If our use-case strictly has an input (customer email) and an output (reply), a seq2seq model is suitable because it's designed to take an input sequence and produce an output sequence. Models like **BART** (which was pre-trained on a mix of tasks including summarization) or **T5** can work well. Alternatively, using a decoder model like GPT-2 is possible: we would concatenate the input and output with a separator and train it to continue from input to output.

**Decision:** Let's use a sequence-to-sequence approach with a model like `t5-small` or `BART-base` for demonstration, since they can handle input -> output mapping naturally. If our data is not too large and we

want fast experiments, `t5-small` (60M parameters) is good. For better results and if more compute is available, `t5-base` (220M) or `bart-base` (140M) could be used similarly.

### 9.3 Preparing the Dataset

Say we have a dataset of support queries and responses. For illustration:

**Customer Query:** "Hello, I received the wrong item in my order. I ordered a blue shirt, but got a red one. How can I exchange it?"

**Support Answer:** "Hi there! I'm sorry about the mix-up. Don't worry, we'll fix this. Please provide your order number, and we'll send you the correct shirt immediately. We'll also arrange a pickup for the wrong item. Thank you for your patience!"

We need to create training data in a format the model can learn from. For T5, we might prepend a prefix like `"reply:"` to the query to indicate the task (this is optional but can help if using the T5 architecture which expects a task prefix). So input to model: `"reply: Hello, I received the wrong item in my order...?"`, target: the support answer.

**Preprocessing steps:** 1. Tokenize the input and output. Use T5 tokenizer (which will likely prefix tokens for the input and output appropriately). 2. Ensure the sequences are within length limits (T5-small might allow input up to 512, output up to 128 or so by default configuration). 3. Possibly filter or shorten extremely long queries, or use truncation.

We'd do this for all Q&A pairs. Tools: we could use a CSV of queries/answers and the `datasets` library to load and tokenize, similar to what was shown in earlier chapters.

### 9.4 Fine-Tuning with Hugging Face (Code Example)

We will outline the code without necessarily running it here, but this is how it could look:

```
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM, Trainer,
TrainingArguments

# Load dataset - assume we made a dataset script or have a local file
data = load_dataset('csv', data_files={'train': 'support_train.csv',
'validation': 'support_val.csv'})

model_name = 't5-small'
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

# Preprocess function
def preprocess_batch(examples):
    inputs = ["reply: " + q for q in examples['query']]
    model_inputs = tokenizer(inputs, max_length=256, truncation=True)
```



```

# tokenize targets
with tokenizer.as_target_tokenizer():
    labels = tokenizer(examples['answer'], max_length=128, truncation=True)
model_inputs['labels'] = labels['input_ids']
return model_inputs

tokenized_data = data.map(preprocess_batch, batched=True,
remove_columns=data['train'].column_names)

training_args = TrainingArguments(
    output_dir="support-model",
    evaluation_strategy="epoch",
    learning_rate=5e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=3,
    save_strategy="epoch",
    logging_steps=100,
    logging_dir="logs",
    report_to="none" # to keep it simple; could use 'wandb' or 'tensorboard'
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_data['train'],
    eval_dataset=tokenized_data['validation'],
    tokenizer=tokenizer
    # We can add compute_metrics here if we want to measure something like BLEU
    or ROUGE on validation
)

trainer.train()

```

This would fine-tune T5-small on our support Q&A. Each epoch it will evaluate on the validation set (if given). After training, the final model weights (likely the last checkpoint) are in `support-model` directory.

We should check a couple of sample outputs: take a query from validation or a new one and see:

```

test_query = "Hi, my internet is down since this morning. Is there an outage in my area?"
input_ids = tokenizer("reply: " + test_query, return_tensors='pt').input_ids
outputs = model.generate(input_ids, max_length=100, do_sample=False)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))

```

This might output something like a plausible support reply (depending on how well it learned style): e.g. "Hello! Sorry to hear about your internet issues. Let's get that sorted out. It appears there is a known outage in your area since 8 AM. Our team is working on it and we expect service to be restored by noon. Thank you for your patience!" – or something along those lines if the model learned to be apologetic and helpful.

**Training Considerations:** With 8 batch size and T5-small, 3 epochs on a few thousand examples might finish in tens of minutes on a GPU. If the dataset is large (say 50k examples), maybe 1-2 epochs might suffice with an early stopping.

## 9.5 Evaluating the Generation Model

Unlike classification, automatic evaluation is tricky. We could use metrics: - **BLEU** or **ROUGE** if we have reference answers (which we do). These metrics check overlap between generated answer and the reference answer. They are not perfect (they might penalize a perfectly good answer that uses different phrasing), but they can catch egregious problems or measure improvement over time. - **Human Evaluation:** Ultimately, especially for customer support, you'd have humans review some outputs. Check: Are they polite? Correct? On brand? Did the model hallucinate information? (E.g., it should not make up a fake outage time if not sure – this is an issue: models tend to *always* answer confidently. If training data has a pattern of always giving some answer, the model will do so even if it doesn't actually *know* in a real situation. We might need to include examples where the support says "I don't have that information" appropriately, if that's desired. This is part of aligning model behavior.)

**Avoiding Pitfalls:** We should inspect if the model started copying queries into answers or something odd. If training examples always had a certain structure, the model might stick to it. That's fine if the structure is good, but watch out for: the model might sometimes produce verbatim training answers if a query is similar (which could be fine since presumably that answer is appropriate). We want to ensure no leakage of private data: if the training had actual customer names or order numbers, the model could inadvertently regurgitate them in similar contexts – a reason to possibly anonymize data or use general placeholders in training.

## 9.6 Fine-Tuning for Creative Generation

Let's briefly mention the case of style emulation (like fine-tuning GPT-2 on Shakespeare works or on a certain author's style). This is *unconditional generation* or *continuation generation*. Process: - Gather a corpus of text in the target style (e.g., collected works of Shakespeare). - Use a **causal language model** (GPT-2, etc.). Prepare the text: you might just concatenate all, or present them in chunks. - Fine-tune GPT-2 on that text (basically language modeling – no special input other than context). This can be done by treating each training example as a segment of the text. - The model will adapt to the style, vocabulary, and content of that corpus. - Then, generation: you prompt it with a line or just start generation and it will produce text likely resembling Shakespearean language.

**Real-Life Example – Fiction Author Assistant:** Authors sometimes want an AI to write in their style. One approach is fine-tuning a model on all their past writings. For example, feed in all novels of a certain author to a model like GPT-J (6B) fine-tune it. Then prompt: "Write a new story about a wizard school." The output might sound like that author's voice, which could help the author draft new material. There are ethical questions (e.g., if the author is fine-tuning on themselves or an assistant on someone else's style, etc.), but technically it's the same pipeline.

## 9.7 Using OpenAI API for Custom Generation

If we switch perspective to OpenAI: We could accomplish the support answer task by fine-tuning GPT-3.5 through their API. Steps: - Create a training file in JSONL. Each line:

```
{ "messages": [
  { "role": "user", "content": "<customer query>" },
  { "role": "assistant", "content": "<support answer>" }
]
```

We'd do that for thousands of Q&As.

- Use `openai` Python library or CLI:

Upload: `openai.File.create(file=open("train.jsonl", "rb"), purpose="fine-tune")` (for GPT-3 style) or `purpose="fine-tune"`, maybe for chat models they had a different endpoint but likely similar. The uploaded file returns an `id` like `file-abc123`.

Create fine-tune: `openai.FineTuningJob.create(model="gpt-3.5-turbo", training_file="file-abc123", validation_file="file-xyz789", suffix="supportbot")`.

- Monitor the job. They might provide events logs through API or you poll status. After it's done (maybe an hour or more, depending on data size and model), you get a fine-tuned model name, something like `ft:gpt-3.5-turbo:your-org:supportbot:xyz`.

- Use it:

`openai.ChatCompletion.create(model="ft:gpt-3.5-turbo:your-org:supportbot:xyz", messages=[{"role": "user", "content": "Hello, I got a wrong item..."}])`. The response should follow the fine-tuned style.

One advantage: OpenAI's base model is powerful, so even with moderate data, it will likely pick up the style quickly and maintain general knowledge. A disadvantage: you can't see or tweak the model after – it's on their server. Also, if you want to fine-tune further or do iterative experiments, each run costs money.

OpenAI fine-tuning in their docs has tips like use a diverse set of prompts, don't have too long outputs, and that you can train for a certain number of epochs (default might be 1-3). They also report training loss.

**Quality Check:** If the fine-tuned OpenAI model misbehaves (maybe it starts giving too terse answers or too verbose if the data was inconsistent), you might refine the dataset or adjust instructions. Sometimes, including a system role in training (like a system message that says "You are a helpful support assistant") might help if provided during inference, even if not explicitly fine-tuned.

**Real Example – Custom Tone:** A company might fine-tune GPT-3.5 to talk like their brand persona. E.g., a playful tone with emojis in social media replies. They provide examples of how they'd respond on Twitter to praise, complaints, etc. After fine-tuning, the model can consistently produce on-brand responses which the social media manager can use as drafts.

We have now covered how to fine-tune for generation with both an open-source approach and the OpenAI API approach. Next, we will move to a different kind of task: text classification fine-tuning.

## Chapter 10: Fine-Tuning for Text Classification

Text classification is a common application of NLP, and fine-tuning an LLM (or rather, a smaller language model) for classification tasks can significantly boost performance over out-of-the-box methods. In this chapter, we'll look at how to fine-tune a model to classify text into categories. Examples include sentiment analysis, topic categorization, spam detection, etc. We'll give a practical guide using Hugging Face, and touch on OpenAI classification fine-tuning.

### 10.1 Task Definition – Classification Scenarios

For concreteness, let's pick **sentiment analysis** on movie reviews as our running example. The task: Given a movie review text, classify the sentiment as Positive or Negative (binary classification). We could also do multi-class (like positive/neutral/negative or star ratings), but let's keep it binary sentiment for simplicity.

This is a supervised learning task. We need a labeled dataset of movie reviews. A classic one is the **IMDB reviews dataset** (25,000 training, 25,000 test, with positive/negative labels). Another is SST-2 (Stanford Sentiment Treebank). We'll assume we have such data.

### 10.2 Choosing a Base Model for Classification

While any LLM could theoretically be fine-tuned for classification, typically we use models that have been pre-trained with a focus on understanding text. **BERT** and its variants (RoBERTa, DistilBERT) are very popular for classification tasks. They have an `[CLS]` token representation often used for classification heads. Alternatively, generative models like GPT can be used by adding a classification head or by prompting (like asking it to output a label), but fine-tuning GPT-2 for classification is less straightforward than using BERT. So we'll opt for a BERT-based model.

Let's say we use `bert-base-uncased` (110M parameters). It's a good trade-off of performance and size. We could use something smaller like `distilbert-base-uncased` (66M) for faster training with slight accuracy hit. The Hugging Face `AutoModelForSequenceClassification` for BERT will automatically add a classification layer on top (a linear layer) if not already present <sup>14</sup>. In BERT's case, it typically replaces the pre-trained next-sentence or MLM heads with a new uninitialized one for our labels (this is indicated by warnings when loading such a model <sup>38</sup>).

### 10.3 Data Preparation

The movie reviews dataset typically has texts and a label (0 for neg, 1 for pos, or vice versa). We will: - Load the dataset (Hugging Face has `load_dataset('imdb')` readily available which gives train/test splits). - Tokenize the text. We need to be careful with length: some reviews can be long, but BERT has a limit (512 tokens). We will truncate longer reviews to 512 tokens. It might slightly degrade performance if key info is at the end of truncated text, but it's usually okay or one can do smarter splitting if needed. - Map labels to numeric if not already (IMDB probably gives 0/1 already). - Possibly balance classes if they were skewed, but IMDB is balanced (25k pos, 25k neg). If you had a skewed dataset, you might consider stratified sampling or weighting the loss function (HF's Trainer can accept `class_weights` indirectly via custom loss, but often not needed unless extreme imbalance).

### Preprocessing Example:

```
from datasets import load_dataset
from transformers import AutoTokenizer

dataset = load_dataset("imdb")
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

def tokenize_fn(examples):
    return tokenizer(examples['text'], padding="max_length", truncation=True,
max_length=256)
    # using max_length=256 to speed up, or 512 for full length

tokenized = dataset.map(tokenize_fn, batched=True)
# The labels are probably already in dataset['label'] as 0/1. Ensure our
columns:
tokenized = tokenized.rename_column("label", "labels")
tokenized.set_format("torch", columns=["input_ids", "attention_mask", "labels"])
```

We choose `max_length=256` for efficiency; many reviews will fit in that, some will be truncated. We use `padding="max_length"` to pad shorter reviews to 256 too (so we can use a static shape, though Trainer can also dynamically pad if you prefer).

## 10.4 Fine-Tuning with Hugging Face Trainer

Similar to previous chapter, we'll use Trainer:

```
from transformers import AutoModelForSequenceClassification, Trainer,
TrainingArguments

model = AutoModelForSequenceClassification.from_pretrained("bert-base-uncased",
num_labels=2)

training_args = TrainingArguments(
    output_dir="sentiment-model",
    evaluation_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=3,
    weight_decay=0.01,
    logging_steps=50,
    logging_dir="logs"
)
```

```

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized['train'],
    eval_dataset=tokenized['test'],
    # using test as eval for simplicity (or one could split train into train/val)
    tokenizer=tokenizer,
    compute_metrics=compute_metrics # we'll define accuracy
)

# Define a simple accuracy metric function
import numpy as np
from datasets import load_metric
accuracy_metric = load_metric("accuracy")

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    return accuracy_metric.compute(predictions=preds, references=labels)

```

We set a typical BERT learning rate (2e-5 is common for BERT fine-tuning, often between 2e-5 and 5e-5). Using weight\_decay to reduce overfitting a bit. We log every 50 steps to see progress.

The `compute_metrics` uses an `accuracy` metric from Hugging Face's evaluate library to return accuracy on the eval set.

Now run `trainer.train()`. On a single GPU, BERT on IMDB (25k training examples, batch 16) for 3 epochs will take some time (maybe an hour or two on a good GPU). But should reach ~90%+ accuracy if done right (BERT fine-tuned on IMDB can hit ~93% typically).

During training, we'd see logging like: step, loss, etc., and after each epoch an evaluation accuracy is printed.

We might see something like: Epoch 1: train\_loss ~ 0.2, val\_accuracy ~ 0.90; by epoch 3 perhaps val\_accuracy ~ 0.92. If eval accuracy starts dropping, we might have overfitted, but usually 3 epochs is fine for this size.

## 10.5 Evaluating and Using the Classifier

After fine-tuning, we have `sentiment-model` directory with the model. We can load it and test on new text:

```

text = "I fell asleep halfway through the film. What a waste of time."
inputs = tokenizer(text, return_tensors='pt', truncation=True, max_length=256)
outputs = model(**inputs)
logits = outputs.logits
pred = logits.argmax(dim=-1).item()

```

```
label = "positive" if pred == 1 else "negative"
print(label)
```

This should print "negative". We can test a positive review similarly.

The model will output a logits tensor of shape (1,2) for the two classes. We took argmax to get the predicted class index.

If we want probabilities, we can apply softmax to logits. For example, `probs = torch.nn.functional.softmax(logits, dim=-1)`. Then `probs[0][1]` might be 0.95 meaning 95% positive, etc.

**Real-World Integration:** Once the classifier is working, it could be deployed as a microservice or embedded in an app. Hugging Face's `pipeline` can simplify usage:

```
from transformers import pipeline
sentiment_pipeline = pipeline("sentiment-analysis", model="sentiment-model")
result = sentiment_pipeline("The movie was a dazzling spectacle with a mediocre story.")
# result will be like [{'label': 'positive', 'score': 0.85}]
```

This pipeline will automatically handle tokenization and output a friendly format.

## 10.6 OpenAI API Fine-Tuning for Classification

OpenAI models (like the base GPT-3 Ada, Babbage, etc.) can be fine-tuned for classification as well, though classification might also be done by prompting (like: "Review: ... Sentiment?" and the model answers "Positive"). But for higher accuracy or if you want a stable, specific classifier, fine-tuning is an option.

OpenAI had an example where they fine-tuned a model to return a category directly. The process: - Each training example prompt is the text, and completion is the label. E.g., `{"prompt": "Review: I loved this movie! It was awesome.\nSentiment:", "completion": " Positive"}` and similarly a negative example with " Negative". - Note: Typically you put a distinctive separator before the completion in training (like a space or newline) so the model knows where prompt ends and completion begins. The OpenAI docs often show prompt ending with `\n\n###\n\n` or something and completion starting with a space, etc. But a simpler scheme works too.

- Then fine-tune e.g. `davinci` on this file. After fine-tuning, you can query:

```
openai.Completion.create(model="curie:ft-yourorg-...", prompt="Review: ...
\nSentiment:", stop=["\n"])
```

 and it will likely complete with " Positive" or " Negative".

Alternatively, OpenAI provides a classification endpoint that could use embeddings or few-shot. But focusing on fine-tune: it essentially teaches the model to map text to label. The advantage is speed and straightforward output, but note: using a giant model to just do classification might be overkill except if you

already have it. Actually, OpenAI's smallest model Ada is quite cheap and after fine-tuning can be used for classification tasks with decent results if given enough examples.

**Case Study:** You have a custom classification like categorize support tickets into "billing issue", "technical issue", "account management", etc. You can fine-tune a model on a few hundred labeled tickets. After that, given a new ticket description, the model can output directly "Technical issue". This might be easier for some than building a whole BERT pipeline, especially if your data isn't huge and you prefer the convenience of an API.

One thing: for classification, OpenAI's newer approach might be using GPT-3.5 with few-shot prompting instead of fine-tuning, since GPT-3.5 is quite capable zero or few-shot. But fine-tuning ensures consistent format and potentially better performance on narrow classes.

## 10.7 Multi-Label Classification and Other Variants

So far, we did single-label classification (one label out of a set). If you have multi-label (e.g., an email can be marked with multiple applicable tags), you'd need to adapt: - One way is to treat it as multiple binary tasks (one per tag) and fine-tune accordingly or use a sigmoid output with a multi-label loss (BCE loss). - Hugging Face's `AutoModelForSequenceClassification` can be adapted by specifying `problem_type="multi_label_classification"` in the config which will use BCE loss instead of softmax, allowing non-exclusive labels. - Or you can fine-tune separate models per label if that's easier, though combined is usually better.

**Example:** Classifying news into topics where a news article might fall in multiple categories (e.g., "politics" and "economy"). Multi-label fine-tuning can handle that.

## 10.8 Evaluating Classifiers

Key metrics: - Accuracy (for balanced classes). - Precision, Recall, F1 (especially if classes imbalanced or if you care about one class more e.g., detecting spam vs ham where spam is rare, you care about recall of spam). - Confusion Matrix to see what gets confused.

In our sentiment example, accuracy suffices since balanced. If we had 5-star ratings, we could measure something like exact match accuracy or maybe correlation with actual, etc.

Our training code already computed accuracy. We could easily add computing Precision/Recall via `load_metric("precision")` etc., or directly use `sklearn.metrics` if needed in a custom compute.

For OpenAI, you might manually evaluate by sending test examples to the model (they didn't have an auto eval during fine-tune, aside from giving you training loss). So you'd query the fine-tuned model on a test set and measure accuracy by comparing to known labels.

## 10.9 Real-Life Example – Spam Detection

Consider an email provider fine-tuning a model to classify spam vs not spam. - They gather thousands of emails with spam/ham labels from their system. - They choose a model like DistilBERT or even a smaller custom model, since speed is crucial for email filtering (maybe they can't use a huge model per email). -



They fine-tune and deploy it such that each incoming email goes through this model. If predicted spam, it gets filtered. - They would continuously evaluate its performance and might update the fine-tuning regularly with new spam examples (since spammers evolve tactics). - They might also integrate the model's output with rule-based systems (for instance, if model is uncertain but an email contains known bad links, you still block it).

Compared to older approaches (which used manual rules or simpler keyword-based machine learning), a fine-tuned transformer might catch more subtle spam (e.g., based on writing style or context) with fewer false negatives.

## 10.10 Conclusion of Classification Chapter

Text classification is a bread-and-butter task where fine-tuning shines. With a relatively small dataset (often a few thousand labeled examples), you can achieve high accuracy. Pre-trained models give a big head start – our BERT example needed only a few epochs and got great accuracy, whereas training a classifier from scratch on 25k IMDB reviews would be much harder (and likely perform worse, given the limited data relative to language complexity).

We also saw that using the OpenAI API is an alternative if you prefer not to handle the training process yourself and don't mind using their service.

Next, we'll move on to fine-tuning for *chatbot style* interactions, which is a bit more complex due to the conversational nature and sometimes requiring advanced techniques like RLHF for best results.

## Chapter 11: Fine-Tuning for Conversational AI (Chatbot Style)

Building a chatbot that can converse naturally, follow instructions, and stay context-aware is a multi-faceted challenge. In this chapter, we cover how to fine-tune an LLM to behave in a conversational or **instruction-following** style. We will discuss supervised fine-tuning for dialogue and touch upon the more advanced Reinforcement Learning from Human Feedback (RLHF) that was famously used to train ChatGPT <sup>39</sup>.

### 11.1 What Do We Mean by "Chatbot Style"?

A "chatbot style" model is one optimized for **multi-turn interactions** and usually **instruction following**. This means: - The model receives input in a conversational format (with a history of user messages, possibly system and assistant messages). - The model should output an answer that is appropriate given the context, often in first person as the assistant. - Ideally, the model should handle a wide range of requests (answer questions, follow commands, ask clarifications, etc.) while maintaining a consistent persona or tone as needed.

OpenAI's ChatGPT (based on GPT-3.5/4) is the prime example: it's an LLM fine-tuned to have a conversational style, refuse certain inappropriate requests, and give helpful answers. That was achieved through a combination of supervised fine-tuning on demonstrations and RLHF <sup>39</sup>.

For our practical approach, we'll start with **supervised fine-tuning**: using dialogue data to teach a model how a conversation goes.

## 11.2 Preparing Dialogue Data for Fine-Tuning

We need training examples of dialogues. There are a few types: - **Human-human conversations** (like transcripts of chats, e.g., from customer support logs or open dialogue datasets). - **Human-AI or AI-AI synthetic conversations** (like the Alpaca dataset from Stanford, which was generated by GPT-3 to simulate instruction-response pairs). - **Instruction-response datasets** where each example is a user instruction and a desired response (even if not exactly a conversation, these single-turn Q&A pairs can be strung into dialogue format).

**Format:** If using a causal language model (like GPT-2 family or LLaMA), a common format is to serialize the conversation into one text with special tokens or delimiters for each turn. For example:

```
<|user|>: How do I bake a chocolate cake?
<|assistant|>: To bake a chocolate cake, you'll need flour, sugar, cocoa powder,
and...
<|user|>: Great, and how long should I bake it?
<|assistant|>: You should preheat your oven to 350°F and bake the cake for about
30 minutes...
<|end|>
```

At training, the model sees the full conversation and tries to predict it (with the loss usually only applied on the assistant parts to avoid training it to generate user questions). There are nuances: you might mask user turns in loss or use a special label to only compute loss on assistant text.

If using an **encoder-decoder** (like T5 or a dialogue model like BlenderBot), you might feed the conversation history into the encoder and have the decoder generate the next reply. But often, large chat models are decoder-only for efficiency with long histories.

For simplicity, consider focusing on one-turn instruction-following: that reduces to input: instruction, output: assistant response. That is exactly like we did in text generation, except with an emphasis on a polite, helpful style and potentially a diverse set of tasks.

**Example single-turn dataset (like Alpaca style):**

```
### Instruction:
Write a friendly welcome message for a new user joining an online forum.

### Response:
Hello and welcome! We're excited to have you in our community. Feel free to ask
questions or share your thoughts. Everyone here is very friendly, and we can't
wait to hear more from you. Enjoy your time here!
```

Many such examples across different tasks (from writing code, explaining jokes, etc.) can be used to fine-tune.

**Multi-turn data example:** Another source is the **PersonaChat** or **DailyDialog** dataset where conversations are multi-turn. PersonaChat includes persona info (which could be like a system prompt telling the model who it is).

We should preprocess conversations to a single text sequence if using a causal LM. Alternatively, one can train with the chat format natively by customizing the model or using a special architecture with multiple inputs (some research models do that). But let's assume standard approach: convert to one sequence with markers.

### 11.3 Using a Pre-trained Base Model Suited for Chat

If possible, start from a model that's already known to be good at text generation and reasonably large for coherence. e.g.: - **GPT-2** or **GPT-Neo** family for open source. GPT-Neo 2.7B or GPT-J 6B are open models that can be fine-tuned, though 6B might need GPU multi or 8-bit. - **LLaMA** or **LLaMA-2** (if you have access and the license fits) – those are powerful and training data included a lot of conversations perhaps. Actually LLaMA was not instruction tuned by default (LLaMA-2 has a chat version out-of-box). - There are smaller instruct models already (like `t5-v1_1-x1` fine-tuned on dialogue, or FLAN-T5 which was instruction tuned on many tasks). Starting from something like `google/flan-t5-x1` (3B) might give a head start because FLAN-T5 was instruction tuned on a variety of tasks by Google. Fine-tuning that further on chat data could yield good results with fewer epochs.

For learning purposes, we might use something like `facebook/blenderbot-400M-distill` (which is a 400M parameter dialogue model pre-trained on Facebook's dialogue tasks) or a smaller GPT-2.

However, fine-tuning GPT-2 (117M) on a complex chat might not yield great quality due to limited capacity. But it will show the mechanics: - We'll use a causal LM, format the input as a conversation, and train it to predict the next response given the context.

### 11.4 Fine-Tuning Process

**Data Format Decision:** Let's decide to do single-turn instruct fine-tuning first (for simplicity), which is basically Q -> A pairs, one per example, similar to earlier generation tasks but focusing on instructions.

We create input text as, say, `"User: <instruction>\nAssistant:"` and target as the assistant's response. Or we can incorporate it in one sequence and rely on the model to generate after "Assistant:".

Better approach: Provide the model with prompt including user query and some signal that it's the assistant's turn, then only train it on writing the assistant's part.

One way in code: in the concatenated sequence, you mask out loss for the prompt portion. Hugging Face's Trainer allows specifying `labels` such that positions of the input prompt are labeled `-100` (which are ignored in loss computation by default).

So we could do:

```
"<s>User: How do I bake a cake?\nAssistant: "
```

And then the rest of the sequence after "Assistant:" is the answer. We'll create `input_ids` for the whole thing, and labels that are same as `input_ids` but with `label = -100` for everything up to "Assistant:" (so the model only gets loss on generating the answer).

Alternatively, we can shift approach: For training, feed the entire prompt as context and just have the model generate the answer as target sequence. This is easier with encoder-decoder (like feed the conversation into T5 encoder, have decoder target the answer). But with GPT-2, you have only one stream.

Let's illustrate with a pseudocode for GPT-2 fine-tune:

```
from transformers import AutoTokenizer, AutoModelForCausalLM, Trainer,
TrainingArguments
data = load_dataset("path_to_your_instruction_data") # assume it gives
'instruction' and 'response' fields

model_name = "gpt2-medium"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name)

# Ensure tokenizer can use special tokens we want
user_token = "<|user|>"
assistant_token = "<|assistant|>"
tokenizer.add_special_tokens({'additional_special_tokens': [user_token,
assistant_token]})
model.resize_token_embeddings(len(tokenizer))

max_length=128 # max combined length of prompt+response we expect
def build_dialogue(ex):
    prompt = f"{user_token}: {ex['instruction']}\n{assistant_token}:"
    prompt_ids = tokenizer(prompt, truncation=True, max_length=max_length,
add_special_tokens=False)['input_ids']
    answer_ids = tokenizer(ex['response'], truncation=True,
max_length=max_length, add_special_tokens=False)['input_ids']
    # Create labels: ignore prompt part
    labels = [-100]*len(prompt_ids) + answer_ids + [tokenizer.eos_token_id]
    input_ids = prompt_ids + answer_ids + [tokenizer.eos_token_id]
    # Also create attention mask
    attention_mask = [1]*len(input_ids)
    return {"input_ids": input_ids, "attention_mask": attention_mask, "labels":
labels}

train_data = data['train'].map(build_dialogue)
val_data = data['validation'].map(build_dialogue)
```

```

training_args = TrainingArguments(
    output_dir="chatbot-model",
    per_device_train_batch_size=4,
    num_train_epochs=2,
    learning_rate=5e-5,
    gradient_accumulation_steps=8, # if batch 4 to simulate batch 32
    fp16=True, # if using GPU with half precision support
    evaluation_strategy="steps",
    eval_steps=500,
    save_steps=500,
    logging_steps=100
)
trainer = Trainer(model=model, args=training_args, train_dataset=train_data,
    eval_dataset=val_data)
trainer.train()

```

This is a bit lower-level than using a `TextDataset` but illustrates constructing custom label masking.

We added special tokens for user and assistant roles. We appended an `eos_token` at end of each sequence to denote end-of-turn or end-of-conversation, which GPT-2 uses as a stop.

We must `.resize_token_embeddings` because we added tokens to tokenizer (so model's embedding matrix and output matrix are expanded).

This training would teach the model that after seeing `<|user|>: ...<|assistant|>:` it should output something appropriate until EOS.

**Alternatively**, if using an instruct-tuned model like `flan-t5`, we could just feed it as a normal seq2seq: input = instruction, output = response, which is simpler. But let's stick to one approach.

## 11.5 Reinforcement Learning from Human Feedback (RLHF) Overview

While the above supervised fine-tuning can get a model that gives decent responses, it might still be suboptimal in terms of alignment (might not exactly follow what users prefer). RLHF was used to further align ChatGPT: - First, a **reward model** is trained: humans rank different model responses to a variety of prompts from best to worst <sup>39</sup>. The reward model (a separate network, often same architecture as the language model but outputs a scalar) is trained to predict higher score for the more preferred outputs <sup>39</sup>. - Then the language model (policy) is further fine-tuned with **reinforcement learning (e.g., Proximal Policy Optimization, PPO)** using the reward model as a feedback signal. The model generates a response, the reward model scores it, and the PPO algorithm adjusts the model to maximize reward while not straying too far from the original model (they add a KL penalty to keep it from diverging and just outputting some high reward gibberish) <sup>40</sup>.

RLHF tends to make the model more *polite, correct, on-topic*, etc., because the human raters would have rewarded those qualities. It's an advanced step that requires quite some setup (human annotations or at

least a proxy reward, and implementing a RL loop). There are libraries for this: e.g., Hugging Face's **TRL** (Transformers RL) library has implementations of PPO for language models.

For someone reading this book, it's good to know conceptually. Actually doing RLHF fine-tune might be out of scope, but if one is adventurous: - They might try using a smaller model and a simple reward like "avoid toxic content, etc." just for experiment.

**However, for practical purposes**, a lot can be achieved just with supervised fine-tuning, especially if you incorporate a diverse, well-curated set of examples of desired behavior.

## 11.6 Handling Long Conversations and Memory

A fine-tuned model for chat needs to carry conversation history. In our approach, the entire relevant history is fed each time. This is feasible if the conversation is short or we have a large context window (like GPT-3.5 can handle ~4k tokens of context, GPT-4 even 8k-32k in some versions).

For smaller models like GPT-2, 1024 tokens is max context, so it might forget older messages if conversation goes beyond that. Some strategies: - Use summarization or state tracking: e.g., compress earlier turns into a brief summary and include that in prompt for long chats. - Limit scope: e.g., always assume only last N turns matter for context (works if conversation branches aren't too long).

## 11.7 Testing the Fine-Tuned Chatbot Model

After training (maybe 2 epochs on, say, 50k instruction-response pairs), we test: - Provide a prompt conversation (simulate how inference would happen). Using our format: Input: "<|user|>: Could you explain the theory of relativity in simple terms?\n<|assistant|>:"

We feed that and generate text until an EOS. The model should produce something about theory of relativity simply explained.

Important: we must use `tokenizer.decode` carefully to not include the prompt in output, etc. Or just let it generate after the prompt. In generation, one can specify `stop` tokens (like the special token for user perhaps if it tries to start user role) or rely on EOS.

**Quality check:** - Does the model follow instructions properly? - Does it avoid saying unsafe or incorrect things? (That's tricky; if our fine-tune data included some refusals and corrections, it might). - Usually, a purely supervised model will try to always comply. It might not know when to refuse (like if asked for something harmful, unless that scenario was in training). - It might also hallucinate facts or code if it wasn't corrected via RLHF or external tools – that's a known limitation; e.g. it may explain relativity but might make some errors if underlying model doesn't know or training data had inconsistent info.

**Comparison:** If you have used ChatGPT, you will find the purely supervised fine-tuned (say, the Alpaca model that replicates some of GPT-3.5's instruction following) is decent but not as fine-grained in judgment. It might also lack some knowledge breadth if the base model was smaller or older (e.g., fine-tuning LLaMA-7B on instructions yields a capable model, but it might still fall short of GPT-3.5).

## 11.8 Example: Domain-Specific Chatbot

Imagine we want a chatbot specifically for *medical advice* (just a hypothetical; careful: giving medical advice has risk). We could fine-tune on a dataset of medical Q&A (like WebMD FAQs or doctor-patient dialogues). The model then would hopefully pick up the formal tone and the practice of suggesting seeing a doctor for serious issues, etc.

Or a **customer support bot** for a product: fine-tune on past support chat transcripts. Then new customers can chat with this model to troubleshoot issues. The model speaks in the company's style and knows the common solutions (because it saw them in transcripts). It's essentially knowledge transfer via fine-tuning.

One must ensure privacy if using real transcripts – maybe scrub personal data or aggregate. Also, updating the bot when product changes is necessary (ongoing fine-tune or provide a mechanism to feed updated info via retrieval, which is outside pure fine-tune).

## 11.9 OpenAI API Approach for Chatbot

As noted earlier in text generation, OpenAI fine-tune for GPT-3.5 would involve conversation examples. Actually for GPT-3.5, they explicitly launched fine-tuning to allow custom behavior while still maintaining a lot of base capabilities.

One would prepare data with `messages` as we did. They might also include a system message in each conversation to set a persona if desired.

If fine-tuning GPT-3.5, you get a model that by default behaves in the way you trained it. However, OpenAI retains safety layers; presumably, even fine-tuned models still have to abide by the content guidelines (the base model and the API might refuse or filter extremely bad content).

For instance, if you fine-tuned GPT-3.5 to be a more terse assistant, it will by default give shorter answers than stock GPT-3.5.

You might *not* need RLHF yourself because the base GPT-3.5 already underwent that. In effect, you are just nudging it.

In open-source context, some folks attempt RLHF by themselves (like training a reward model from user preferences is a whole project in itself – though there are open datasets like OpenAI's summarized feedback or Anthropic's HH dataset with human feedback that one could use to train a reward model).

## 11.10 Wrapping Up Chatbot Fine-Tuning

To summarize: - Fine-tuning for chat requires good dialogue data. The quality and diversity of instructions or questions matters (the model will be only as versatile as the data). - It's beneficial to incorporate guidelines into training data, e.g., example conversations where if the user asks something disallowed, the assistant responds with a safe completion ("I'm sorry, I cannot assist with that request."). - Achieving ChatGPT-level results is hard without RLHF, but a fine-tuned model can still be quite useful for many use cases (especially in narrow domains or with lower user expectations). - Always evaluate with real or

simulated conversations to see how the bot handles them, and be ready to refine with additional data or safety rules.

Now that we've explored conversation fine-tuning, the last practical task in our journey is *summarization*, which we will tackle next.

## Chapter 12: Fine-Tuning for Summarization

Summarization is a task where the model reads a longer text and produces a shorter text capturing the main points. Fine-tuning an LLM (especially an encoder-decoder model) for summarization can yield more accurate and coherent summaries than using a general model. In this chapter, we'll guide through fine-tuning for summarization with a focus on using sequence-to-sequence models like T5 or BART.

### 12.1 Task Definition – Summarizing Text

Let's assume we want to summarize news articles into a few sentences. This is a common scenario (like the CNN/DailyMail dataset or XSum dataset, where each article has a summary). Summarization can be **extractive** (copying key phrases, not what we aim for with LLMs typically) or **abstractive** (rephrasing, synthesizing). Modern models like T5, BART perform abstractive summarization.

Challenges in summarization: - Maintaining factual accuracy (the model shouldn't introduce stuff not in the source). - Conciseness and coherence. - Handling long input documents (sometimes longer than model's max input length).

We will focus on a relatively manageable input length so that a model like BART (1024 token limit) can handle it.

### 12.2 Choosing a Model for Summarization

Encoder-decoder models are ideal. Options: - **BART (e.g., facebook/bart-base or bart-large)**: Pre-trained on summarization among other tasks and has an architecture suited to it. - **T5**: Also suitable. T5 was trained in a multi-task way including summarization tasks. It uses task-specific prefixes like "summarize:" <sup>19</sup>. - **Pegasus**: A model by Google specifically pre-trained for summarization with a gap sentence objective. They have pegasus-large trained on news. - **FLAN-T5** or **UL2** etc.: Newer models that incorporate multi-task training, often including summarization, can be fine-tuned further.

Let's pick **BART-base** (about 140M params) for demonstration. It's reasonably powerful and faster to train than BART-large (400M). If we had more resources or needed higher quality, BART-large or Pegasus might be used.

### 12.3 Data Preparation

Assuming we have a dataset of (article, summary) pairs. For example, CNN/DailyMail is large (~300k examples of news). For a quick experiment, maybe use a smaller subset or a smaller dataset.



We need to: - Tokenize articles and summaries. - Possibly use a special separator or just rely on model's encoder/decoder separation. For BART, we feed article into encoder and expect it to generate summary from decoder, so training format with HuggingFace is straightforward using `Seq2SeqTrainer`.

**Important Preprocessing Steps:** - Truncate or split if articles are longer than model can accept (1024 tokens for BART). CNN/DM articles might fit since they are usually not extremely long (some are maybe 800 tokens). XSum articles are short (one paragraph). - We might add a prompt like "Summarize:" at the beginning of input if using T5 (since T5 expects that prefix <sup>19</sup>). For BART, not needed; just give article text. - We should consider that during generation, the model might output more tokens than desired length. We can set a `max_length` for generation and maybe a `no_repeat_ngram` to avoid repetition.

#### Example using Hugging Face Transformers:

```
from datasets import load_dataset
dataset = load_dataset("cnn_dailymail", "3.0.0", split="train[:1%]")
# using 1% for demonstration
dataset_val = load_dataset("cnn_dailymail", "3.0.0", split="validation[:1%]")

from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("facebook/bart-base")

max_input = 512
max_target = 128

def preprocess_function(examples):
    inputs = examples["article"]
    # Optionally prepend something like "summarize: " if using T5.
    model_inputs = tokenizer(inputs, max_length=max_input, truncation=True)
    # Tokenize targets with text_target for seq2seq
    with tokenizer.as_target_tokenizer():
        labels = tokenizer(examples["highlights"], max_length=max_target,
truncation=True)
        model_inputs["labels"] = labels["input_ids"]
    return model_inputs

train_data = dataset.map(preprocess_function, batched=True,
remove_columns=["article", "highlights", "id"])
val_data = dataset_val.map(preprocess_function, batched=True,
remove_columns=["article", "highlights", "id"])
```

We removed columns that we don't need post-tokenization, keeping only `input_ids`, `attention_mask`, `labels`, etc.

We used `examples["highlights"]` because CNN/DM uses "highlights" as the summary.

## 12.4 Fine-Tuning with Seq2SeqTrainer

We can use `Trainer` but specifically `Seq2SeqTrainer` which adds some conveniences for generation evaluation. Actually in recent HF, `Trainer` can handle seq2seq too if we pass `predict_with_generate=True` in `TrainingArguments` for evaluation.

We'll do:

```
from transformers import AutoModelForSeq2SeqLM, Seq2SeqTrainer,
Seq2SeqTrainingArguments

model = AutoModelForSeq2SeqLM.from_pretrained("facebook/bart-base")

training_args = Seq2SeqTrainingArguments(
    output_dir="bart-summarizer",
    evaluation_strategy="epoch",
    learning_rate=3e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=2,
    predict_with_generate=True, # enables generation on eval
    logging_steps=100,
    save_steps=500,
    eval_accumulation_steps=2,
    generation_max_length=128,
    generation_num_beams=4 # use beam search for eval generation
)

def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    # decode ignoring pad tokens
    decoded_preds = tokenizer.batch_decode(predictions,
skip_special_tokens=True)
    # Replace -100 in labels as those are pad tokens in labels
    labels = [[l for l in label if l != -100] for label in labels]
    decoded_labels = tokenizer.batch_decode(labels, skip_special_tokens=True)
    # Compute ROUGE
    rouge = evaluate.load("rouge")
    result = rouge.compute(predictions=decoded_preds, references=decoded_labels)
    # return maybe rougeL or average
    result = {k: round(v, 4) for k,v in result.items()}
    return result

trainer = Seq2SeqTrainer(
    model=model,
    args=training_args,
    train_dataset=train_data,
```

```

        eval_dataset=val_data,
        tokenizer=tokenizer,
        compute_metrics=compute_metrics
    )

trainer.train()

```

This will fine-tune BART on the (small) dataset we provided. The metrics computed will be ROUGE (rouge1, rouge2, rougeL by default) <sup>41</sup> <sup>42</sup>. With 1% data, don't expect great ROUGE, but with full data BART on CNN/DM gets something like ~44 ROUGE-1, ~21 ROUGE-2, ~40 ROUGE-L (these are standard metrics if properly trained).

We set `predict_with_generate=True` to have the trainer do actual text generation on the eval set to compute metrics. We used `beam search with 4 beams` to get typically better (higher ROUGE) summaries than greedy.

We must ensure we installed `evaluate` library (we used `evaluate.load("rouge")` which might need pip install evaluate and rouge\_score package).

## 12.5 Using the Fine-Tuned Summarizer

After training, you can use the model to summarize new articles:

```

test_article = "In a shocking turn of events, the stock market soared to record
highs on Tuesday, etc. ..." # a long text
inputs = tokenizer(test_article, return_tensors='pt', max_length=512,
truncation=True)
summary_ids = model.generate(inputs['input_ids'],
attention_mask=inputs['attention_mask'],
                                max_length=100, num_beams=4, early_stopping=True)
print(tokenizer.decode(summary_ids[0], skip_special_tokens=True))

```

This should output a concise summary. For example, if the article was about the stock market event, the summary might say: "Stocks hit record highs on Tuesday following an unexpected announcement by the Federal Reserve. Investors reacted positively, driving the Dow and S&P to new peaks."

We set `early_stopping=True` so that beam search stops when all beams have ended.

## 12.6 Evaluating Summaries

We primarily use ROUGE as an automated metric. It measures overlap of n-grams between generated summary and reference. A high ROUGE usually means the summary covered similar content. But it doesn't measure factual correctness or coherence beyond overlap.

So: - Check ROUGE from our training logs (the `compute_metrics` gives numbers each epoch). - Possibly check **BLEU** if summarizing very formulaic texts (less common for summary). - Human evaluation: Ideally, read a few summaries, see if they capture the main idea and have no hallucinations or distortions.

One known issue: Summaries can sometimes include facts not in source (hallucination). If our model does that, one approach is to constrain generation or fine-tune with careful data or even use models with a copy mechanism (but encoder-decoder usually learns to copy salient info by itself).

**Better Approach** (outside this book's scope): There are newer methods like adding a constraint that certain keywords from the source must appear in summary, or using a separate fact-checker.

However, in open domain summarization, hallucination risk is relatively low if model is well-trained on that distribution (news summarization models often are quite faithful because of training data size; but on arbitrary text, caution is needed).

## 12.7 Fine-Tuning Tips for Summarization

- **Length Penalty:** during generation, you might use a `length_penalty < 1` to encourage shorter output if the model tends to ramble, or `> 1` if it tends to be too short.
- **No Repeat N-gram:** to avoid repetition loops, one can set `no_repeat_ngram_size=3` or `4` in `generate`.
- **Max Length:** If you set too high, model might start diverging into unrelated text after summarizing main points (because it's forced to generate more). So choosing a reasonable max summary length or use some heuristic (like stop when output has ended a sentence and is above min length).
- **Train Data Quality:** The model is only as good as references. News highlights are generally good. If doing something like summarizing scientific papers, the model might need more careful training or larger models to capture all details.

## 12.8 Real-Life Use Cases

- **News Digest:** A company could fine-tune a model on a corpus of news articles and their summaries (like we did). Then daily, they can feed in news articles to get bullet summaries for a newsletter.
- **Legal Document Summary:** Fine-tune on pairs of legal clauses and plain English summaries (if you have those). The model could help lawyers by summarizing contracts or case files.
- **Meeting Transcript Summarizer:** If you have meeting transcripts with human-written minutes, fine-tune a model to produce meeting minutes. This is harder because transcripts are messy and long (maybe needing chunking or an LLM with large context). But even partial summarization (summarize each agenda segment) could help.

One can also combine summarization with dialogue, e.g., a chatbot that summarises a conversation so far, etc., but that would be a separate pipeline, not a single model.

## 12.9 Summarization with OpenAI API

OpenAI's GPT models are quite good zero-shot summarizers. You can just prompt "Summarize the above text:" and it often does decently. But if you have domain-specific summarization, you could fine-tune.

For example, if summarizing financial reports in a certain format (like always include specific metrics), you could fine-tune GPT-3.5 on examples of those. Then use that fine-tuned model to summarize new reports in that standardized way.

OpenAI fine-tune procedure: prompt = long text (or truncated to 2048 tokens if using older models), completion = summary. But note the context length. GPT-3's older models had 2048 token limit; GPT-3.5 has 4k or 16k in some versions. If text exceeds that, you have to split it or maybe decide not to fine-tune but rather use chunking strategy with base model.

Also, summarization fine-tuning in OpenAI might be limited by token windows, so it's often easier to just instruct the base model if it can handle the text length.

However, if your style of summary is special, a fine-tune could help the model consistently produce it, rather than relying on prompt examples each time.

## 12.10 Conclusion

Summarization fine-tuning allows turning an LLM into a specialized abstractor. We demonstrated using BART. In practice, models like **Pegasus** (trained for summarization) fine-tuned on news can reach high performance.

The techniques and steps – data prep, trainer usage – mirror other tasks but with the specifics of sequence-to-sequence outputs and using generation in evaluation.

With this, we conclude the four practical fine-tuning chapters (generation, classification, dialogue, summarization). Each showed how to adapt general models to tasks with code examples.

## Chapter 13: Advanced Topics and Best Practices in Fine-Tuning

We have covered the basics and practical aspects of fine-tuning across tasks. This chapter will discuss some advanced topics and best practices that cut across these scenarios, helping you become an *expert* in fine-tuning LLMs.

### 13.1 Parameter-Efficient Fine-Tuning (PEFT)

Fine-tuning all parameters of a large model can be memory-intensive and sometimes unnecessary. **Parameter-efficient fine-tuning (PEFT)** techniques aim to modify only a small subset of parameters, or add new small parameters, while leaving most of the pre-trained model intact. This drastically reduces GPU memory usage and training time, and allows one to maintain multiple fine-tuned versions cheaply (since each is basically the base model + a small delta).

Key PEFT methods: - **Adapters**: Introduced by Houlsby et al. (2019), adapters are small feed-forward layers added between existing layers of the model <sup>30</sup>. During fine-tuning, only these adapter weights (and maybe layer norm biases) are trained <sup>43</sup>. Everything else stays fixed. Adapters have an "down-project then up-project" structure (two linear layers with a non-linearity) <sup>44</sup>. They add some parameters (a few percent of original model size typically). They've been shown to achieve nearly full fine-tuning performance in many

cases. - **LoRA (Low-Rank Adaptation):** This method by Hu et al. (2021) inserts low-rank matrices into the model's weight updates <sup>45</sup>. Essentially, for certain weight matrices (e.g., the Q and V in attention), LoRA introduces a decomposition  $\Delta W = A * B$  (where A and B are low-rank) <sup>46</sup>. During training, A and B are the only ones updated (randomly initialized to zero so initial  $\Delta W=0$ ) <sup>46</sup>. This means instead of thousands of parameters in W, you train maybe rankr + rhidden parameters (with r like 4 or 8). For large models, LoRA can reduce trainable params dramatically (e.g., GPT-3 175B with LoRA rank 8 has only ~0.1% of parameters trainable). At inference, the AB can be merged into W for no extra cost <sup>47</sup>. *LoRA is very popular because it's simple and effective. As an example, one can fine-tune a 7B model on a single GPU with LoRA <sup>48</sup>, which normally would be hard with full fine-tuning.* - **Prefix Tuning:** *It keeps model weights unchanged but optimizes a set of virtual tokens (prefix) that are prepended to every input <sup>49</sup>. These virtual tokens have embeddings that are learned such that they steer the model's behavior. You don't modify any weight of the model, just learn some new token embeddings (the prefix length might be like 10-100 tokens). At inference, you attach the prefix embeddings to prompt then normal text tokens after. It's extremely lightweight (like a few thousand parameters only) <sup>49</sup>. This works surprisingly well for some tasks, though for very different tasks from what the model was trained on, it might be less effective than fine-tuning deeper layers.* - **Prompt Tuning (Soft Prompts):**\* Similar to prefix tuning, you learn a set of prompt vectors that, when concatenated to the input, guide the model. This was used with GPT-3 for instance, to personalize it to tasks without changing weights.

Using these methods often involves either modifying the training code or using libraries: - Hugging Face's **PEFT** library provides a unified interface. For example, to use LoRA on a transformers model:

```
from peft import LoraConfig, get_peft_model
config = LoraConfig(r=8, lora_alpha=16, target_modules=["q_proj", "v_proj"],
bias="none", task_type="CAUSAL_LM")
model = get_peft_model(model, config)
```

Then train the `model` normally. The library takes care of freezing other weights and adding LoRA layers. - Results: It's been found that LoRA and adapters often achieve 95-100% of the full fine-tune performance <sup>30</sup> on many benchmarks, at a fraction of training cost. QLoRA (quantized LoRA) even allowed 65B models fine-tuned on a single GPU with minimal loss in quality <sup>33</sup>.

**When to use PEFT:** - If the model is very large and you can't afford full fine-tune memory wise. - If you want to distribute one base model and many small fine-tuned "patches" (like a chatbot that can have multiple personalities loaded as needed). - If you want to reduce overfitting: training fewer parameters can sometimes regularize the model (though not always straightforward). - If you need to swap tasks quickly (adapters can be loaded/unloaded dynamically for multitask systems).

**Real Example:** The Open Source model **Alpaca-LoRA** combined LLaMA-7B with LoRA fine-tuning on instruction data, resulting in a ~4MB LoRA file that, when applied to LLaMA weights, turns it into an instruction-following model. People shared these LoRA weights which are smaller and don't contain original model weights (thus somewhat sidestepping distribution restrictions).

## 13.2 Continual Learning and Catastrophic Forgetting

When fine-tuning, models can sometimes **forget** the original data, especially if new data is small and narrow. This is catastrophic forgetting. For instance, if you fine-tune a language model on only legal texts, it might become worse at casual conversation.

Ways to mitigate: - **Mixing Pre-training Data:** Occasionally, people mix a bit of original general data (or a small sample) with the fine-tuning data to retain knowledge. This is tricky if you don't have the original data or it's huge. - **Regularization:** Techniques like L2 regularization towards original model weights (to not let them drift too far), or EWC (Elastic Weight Consolidation), which tries to slow changes on weights that were important for original tasks. - **Low Learning Rate and Fewer Epochs:** Keeping fine-tuning minimal ensures you're nudging model rather than making it specialize too heavily. - **Adapters/LoRA:** These by design keep original weights and only add new capacity, so the original model's knowledge is still there and can often be recovered by turning off adapters. If you want a model that can multitask, you can keep multiple adapter sets and switch depending on context.

## 13.3 Large Scale Fine-Tuning and Distributed Training

If you have a lot of data or a large model: - **Distributed Data Parallel (DDP):** PyTorch can split batches across GPUs. HuggingFace's Trainer will use multiple GPUs if available. For very large models, **model parallelism** might be needed (splitting layers across devices). Libraries like DeepSpeed or ColossalAI help with that. - **FP16 / BF16 Training:** Using half precision can cut memory usage in half and speed up training on GPUs that support it (A100, V100 etc.). Trainer with `fp16=True` does this (or `bf16=True` on newer GPUs, often more stable). - **Gradient Checkpointing:** Saves memory by not storing intermediate activations; instead, recomputes them on the fly during backprop. Transformer models can reduce memory by ~50% with this at a cost of some compute. Many HF models allow enabling with `model.gradient_checkpointing_enable()`. - **Learning Rate Schedules:** With lots of data, usually you use a scheduler (linear decay, cosine, etc.). HF Trainer by default uses linear schedule with warmup (e.g., warmup 500 steps). Tuning the learning rate schedule can affect stability for very large runs.

## 13.4 Evaluating and Debugging Fine-Tuning

- Always keep an eye on training vs validation loss. If validation diverges from training (i.e., loss stops improving or gets worse while training improves), you might be overfitting – consider early stopping or adding more data augmentation, regularization.
- If training loss doesn't go down at all: maybe learning rate too high (check if loss is NaN or jumping), or data is very noisy/hard, or a bug in data labeling (e.g., wrong labels).
- Use small experiments: try a tiny subset of data and overfit (the model should be able to get near 0 loss on, say, 100 examples if you train long enough). If it can't, something is wrong with setup or data.
- Check output qualitatively. For generation tasks, after fine-tuning, generate a handful of examples to see if it "feels" right. E.g., does your summarizer sometimes copy whole sentences or produce gibberish? Does your chatbot occasionally lapse into a different persona? These clues might show you need to refine data or model choice.

### 13.5 Dealing with Data Limitations

Often, you won't have a massive labeled dataset. Techniques to cope: - **Data Augmentation:** For classification, you can paraphrase existing sentences to create new ones. For dialogue, perhaps augment with some synthetic conversations (maybe using an existing model to generate some). - **Few-Shot Fine-Tuning / Intermediate Fine-Tuning:** If data is scarce, one trick: do an intermediate fine-tune on a related larger dataset first. For example, to build a medical Q&A bot but you only have 1k medical QAs, you might first fine-tune on a general Q&A dataset (like TriviaQA or any large QA corpus) to teach the model the format, then fine-tune on the specific medical data. This way, the model adapts gradually. - **Transfer Learning between tasks:** Sometimes fine-tuning on one task helps another. Multi-task fine-tuning (training on multiple tasks simultaneously) can make a more robust model that you then fine-tune further on a single task.

### 13.6 When Fine-Tuning Might Not Be the Best Solution

We should acknowledge that fine-tuning is not a silver bullet for every scenario: - If you need the model to always incorporate **up-to-date information** (like a QA bot about current events), fine-tuning won't help once the world changes unless you keep fine-tuning constantly. In such cases, using retrieval (search for relevant info) combined with the model (Retrieval-Augmented Generation, RAG) might be better. - If your data is extremely small (say < 50 examples), fine-tuning might just overfit. Using a prompt-based approach or few-shot might generalize better, or consider **Prompt engineering** or using smaller models (that might require less data). - If the base model is already very close to what you need, maybe instruct prompting suffices. E.g., to get GPT-4 to write in pirate speak, you can just instruct it in prompt; fine-tuning GPT-3.5 for that might be overkill (and time-consuming). - If compute is an issue, and the task is not too specific, you might use an existing fine-tuned model from others. The Hugging Face Hub has many models fine-tuned on various tasks – perhaps someone already did sentiment on your language or summarization on similar data. It's worth checking; you could save time by using those or just fine-tuning from that checkpoint rather than the base.

### 13.7 Ethical Considerations in Fine-Tuning

Ethics and bias are crucial: - Your fine-tuned model can inherit or even amplify biases present in the fine-tuning data. Always consider what's in your data: if it's user chats, are there toxic or biased statements in either queries or responses that the model might learn from? Filtering training data for hate, profanity, etc., might be necessary if you don't want your model to produce that. - If fine-tuning for a sensitive domain (law, medicine), be very cautious. The model might produce plausible-sounding but incorrect or dangerous advice. Human oversight or having the model express uncertainty might be needed. OpenAI's models have a lot of work put into refusal when uncertain or out-of-scope; your fine-tuning should also consider that. - Privacy: Fine-tuning on user data (like chat logs, emails) means the model weights now contain info from them, potentially regurgitating it (even if not verbatim, hints could come out). If the data was sensitive, you need to ensure either the model cannot output it or avoid fine-tuning on very sensitive PII altogether. OpenAI's policy on fine-tuning forbids certain personal data types to be used, likely for this reason. - Transparency: Document that a model is fine-tuned and for what purpose, especially if sharing it. Provide appropriate warnings for limitations ("This chatbot is not a certified doctor," etc.).



## 13.8 Future Trends

Looking ahead: - **Larger Contexts:** Models like GPT-4 offer huge context windows. Fine-tuning techniques will adapt to that (maybe fine-tuning to make use of long context efficiently, e.g., summarization of very long texts or multi-document reasoning). - **Multimodal Fine-Tuning:** If models can handle images+text (like GPT-4's vision capability), fine-tuning might involve multimodal data, requiring new strategies (like combining image encoders with LLM decoders). - **AutoML for Fine-tuning:** We may see tools that automatically find the best fine-tuning recipe (learning rate, method, etc.) for a given task. Already, there are efforts to use reinforcement learning or other optimization to tune prompts; similar could come for fine-tuning hyperparameters. - **Continual Learning Systems:** Possibly models that continuously fine-tune on new data streams safely without catastrophic forgetting – an area of active research. - **Alignment Beyond RLHF:** Techniques like **DPO (Direct Preference Optimization)**, which tries to directly fine-tune on comparisons without a reward model, or others might simplify the pipeline for making models aligned with human preferences <sup>50</sup>. It's possible future libraries incorporate these out-of-the-box.

## 13.9 Final Best Practice Checklist

Before ending, let's summarize a checklist when fine-tuning an LLM: 1. **Clarify Objective:** Know your task and pick an appropriate base model (size, architecture). 2. **Curate Data:** Get high-quality, task-specific data. Clean it. If possible, augment or expand it carefully. 3. **Set up Experiment:** Start with a small run (maybe smaller model or subset of data) to verify your pipeline. 4. **Hyperparameters:** Use known good defaults as starting points (e.g., learning rate 2e-5 for BERT, 1e-4 for GPT-2, batch sizes 8-32, etc.). Adjust if needed based on training curve. 5. **Monitor Training:** Watch for divergence or overfitting. Use validation metrics not just training loss. 6. **Evaluate Thoroughly:** Both automatic metrics and manual inspection. Compare with baseline (maybe the base model's output via prompting) to quantify improvement. 7. **Iterate:** If results are not satisfactory, analyze errors. Are certain categories doing poorly? Perhaps add more data there or adjust model (maybe model too small?). 8. **Deploy with Caution:** If deploying, have a way to catch bad outputs (content filter, human review for critical tasks). Monitor user feedback; you might need to fine-tune further as new types of queries come in or as you collect real-world data.

By following these steps and the knowledge from earlier chapters, you should be well-equipped to fine-tune LLMs for a wide range of applications, from basic to advanced.

## Conclusion

We embarked on a journey from the basics of language models to advanced fine-tuning techniques. We learned how LLMs like GPT and BERT are pre-trained and how we can adapt them to specific tasks through fine-tuning <sup>11</sup>. We covered practical implementations for generation, classification, dialogue, and summarization, demonstrating that with the right data and training, a large language model can become a specialist without losing its general language prowess.

Key takeaways: - Fine-tuning leverages existing knowledge in an LLM and hones it for a particular purpose, requiring far less data and compute than training from scratch <sup>13</sup>. - Different tasks have different requirements: sequence-to-sequence models excel at tasks like translation and summarization, while decoder-only models shine in free-form text generation. - Tools like Hugging Face's Transformers and the OpenAI API provide user-friendly pathways to fine-tune models, abstracting much of the complexity. - With

power comes responsibility: fine-tuned models must be used carefully, keeping ethical considerations in mind, and ensuring the outputs are accurate and safe for the end-users.

By understanding both the theory and practice laid out in this book, you should be able to develop your own custom AI models – whether it's a chatbot that carries your company's voice, a classifier that filters information, or a summarizer that digests documents for you. Importantly, you've also learned how to evaluate and iterate on these models.

The field of LLMs is rapidly evolving. Fine-tuning will continue to be a critical tool to unlock the potential of these models, even as new techniques and models emerge. Armed with the knowledge from this comprehensive guide, you won't have to wander around scattered resources – you have a solid foundation from basics to advanced, from which you can dive deeper or tackle new challenges in the world of large language models. Happy fine-tuning!

---

#### 1 Introduction - Hugging Face LLM Course

<https://huggingface.co/learn/llm-course/en/chapter1/1>

#### 2 3 4 5 6 7 8 9 10 11 How do Transformers work? - Hugging Face LLM Course

<https://huggingface.co/learn/llm-course/en/chapter1/4>

#### 12 16 34 35 36 37 How to Fine-Tune GPT-3.5 Turbo With OpenAI API | Scale

<https://scale.com/blog/fine-tune-gpt-3.5>

#### 13 14 15 26 27 28 38 Fine-tuning

<https://huggingface.co/docs/transformers/en/training>

#### 17 18 Fine-tuning with OpenAI. I finally was able to test fine-tuning... | by Practicing DataSci | Medium

<https://medium.com/@j622amilah/fine-tuning-with-openai-624a6055517d>

#### 19 20 21 22 23 24 25 41 42 Fine-Tuning the Pre-Trained T5-Small Model in Hugging Face for Text Summarization | by Yuan An, PhD | Medium

<https://medium.com/@anyuanay/fine-tuning-the-pre-trained-t5-small-model-in-hugging-face-for-text-summarization-3d48eb3c4360>

#### 29 30 33 39 40 43 44 45 46 47 49 Efficient LLM Fine-Tuning: LoRA, QLoRA & RLHF | Medium

<https://medium.com/@hetp2030/parameter-efficient-fine-tuning-of-large-language-models-from-lora-to-rlhf-c17e166da5df>

#### 31 48 Practical Tips for Finetuning LLMs Using LoRA (Low-Rank Adaptation)

<https://magazine.sebastianraschka.com/p/practical-tips-for-finetuning-llms>

#### 32 Parameter-efficient fine-tuning of large language models using ...

<https://www.nature.com/articles/s41598-024-75599-4>

#### 50 Fine-tune large language models with reinforcement learning from ...

<https://aws.amazon.com/blogs/machine-learning/fine-tune-large-language-models-with-reinforcement-learning-from-human-or-ai-feedback/>